

Développement multiplateforme

Flutter

Laurent Bellon 2026-2027

1-1-1 Présentation de l'écosystème : Flutter vs React Native vs Natif

Pour choisir une stratégie de développement mobile, il est nécessaire de comprendre comment chaque approche interagit avec le système d'exploitation (OS) et le matériel.

1. Comparatif des approches techniques

Le choix entre ces technologies repose sur la manière dont le code est exécuté et rendu à l'écran.

Caractéristique	Natif (Swift/Kotlin)	React Native	Flutter
Langage	Swift / Kotlin	JavaScript / TypeScript	Dart
Rendu	Composants natifs de l'OS	Composants natifs via un "Bridge"	Moteur graphique propre (Skia/Impeller)
Performance	Maximale	Bonne (dépend du Bridge)	Excellente (compilé en code machine)
UI	Identique à l'OS	Proche du natif	Dessinée pixel par pixel

Fonctionnement détaillé

Natif

Le code accède directement aux API de l'OS. Il n'y a aucune couche d'abstraction. C'est l'approche la plus performante pour les applications nécessitant un accès intensif au matériel (Bluetooth, capteurs complexes, traitement vidéo).

React Native

Le code JavaScript communique avec les composants natifs via un "pont" (Bridge). Chaque interaction utilisateur doit traverser ce pont, ce qui peut créer des goulots d'étranglement lors d'animations complexes ou de calculs intensifs.

Flutter

Flutter utilise son propre moteur de rendu (Impeller). Il ne demande pas à l'OS de dessiner un bouton : il dessine lui-même le bouton sur un canevas. Cela garantit une cohérence visuelle totale sur toutes les versions d'Android et d'iOS.

2. Cas d'usage professionnels

Développement Natif

- **Exemple** : Application de montage vidéo haute performance ou jeu 3D complexe.
- **Pourquoi** : Accès direct aux API de bas niveau et gestion fine de la mémoire.

React Native

- **Exemple** : Application de e-commerce ou réseau social interne.
- **Pourquoi** : Réutilisation des compétences d'une équipe web (React) et écosystème de bibliothèques JavaScript mature.

Flutter

- **Exemple** : Application bancaire ou outil métier multiplateforme (Mobile, Web, Desktop).
- **Pourquoi** : Besoin d'une identité visuelle forte et identique sur toutes les plateformes, avec une performance proche du natif.

3. Points de vigilance et bonnes pratiques

Erreurs fréquentes à éviter

- **Vouloir tout partager** : Essayer de partager 100% du code entre mobile et web. Bien que Flutter le permette, les contraintes UX diffèrent (clavier physique vs tactile, navigation).
- **Ignorer les spécificités de l'OS** : Même avec Flutter, une application doit respecter les conventions de design (Material Design pour Android, Cupertino pour iOS).

Recommandations

- **Évaluation des besoins** : Si l'application nécessite des mises à jour fréquentes de l'UI sans mise à jour de l'App Store, React Native (via CodePush) offre un avantage.
- **Performance** : Pour des interfaces fluides à 60/120 FPS, Flutter est actuellement la solution multiplateforme la plus robuste grâce à sa compilation AOT (Ahead-of-Time).
- **Maintenance** : Dart est un langage fortement typé, ce qui réduit drastiquement le nombre de bugs à l'exécution par rapport à JavaScript.

4. Synthèse pour le choix technologique

1. **Performance brute** : Natif.
2. **Vitesse de développement & Écosystème Web** : React Native.
3. **Cohérence visuelle & Performance multiplateforme** : Flutter.

Sources

- [Flutter Documentation - Architectural Overview](#)
- [React Native Documentation - Architecture](#)

1-1-2 Architecture technique : Moteur de rendu et langage Dart

La performance de Flutter repose sur deux piliers : son langage de programmation, Dart, et son moteur de rendu graphique.

1. Le langage Dart : Moteur de la logique

Dart est un langage optimisé pour les interfaces utilisateur (UI). Il propose deux modes de compilation distincts selon l'environnement :

- **JIT (Just-In-Time)** : Utilisé lors du développement. Il permet le *Hot Reload*, injectant les modifications de code dans l'application en cours d'exécution sans perdre l'état de l'application.
- **AOT (Ahead-Of-Time)** : Utilisé pour la production. Le code est compilé en instructions machine natives (ARM ou x64), garantissant une exécution rapide et prévisible.

Caractéristiques clés

- **Typage fort** : Détecte les erreurs de type à la compilation.
- **Null Safety** : Le système de type garantit qu'une variable ne peut pas être `null` sauf si elle est explicitement déclarée comme telle, éliminant les erreurs de type `NullPointerException`.
- **Asynchronisme** : Basé sur le modèle `Future` et `async/await`, idéal pour les appels API sans bloquer l'interface.

2. Le moteur de rendu : De Skia à Impeller

Flutter ne délègue pas le rendu des composants aux widgets natifs de l'OS. Il utilise un moteur graphique pour dessiner chaque pixel.

- **Skia** : Moteur historique 2D open-source. Il transforme les instructions Flutter en commandes graphiques compréhensibles par le GPU.
- **Impeller** : Le nouveau moteur de rendu par défaut (sur iOS et progressivement Android). Il résout les problèmes de "jank" (saccades) liés à la compilation des shaders lors de l'exécution en pré-compilant ces derniers.



3. Comparatif des modes de compilation

Mode	Environnement	Avantage principal
JIT	Développement	Hot Reload (productivité)
AOT	Production	Performance native (vitesse)

4. Bonnes pratiques et points de vigilance

Utilisation de l'asynchronisme

Pour éviter de figer l'interface lors d'un appel réseau (ex: récupération de données depuis une API REST), utilisez systématiquement `async/await`.

```
// Exemple : Appel API asynchrone
Future<void> fetchUserData() async {
  try {
    final response = await http.get(Uri.parse('https://api.entreprise.com/user'));
    if (response.statusCode == 200) {
      // Traitement des données
    }
  } catch (e) {
    // Gestion des erreurs réseau
  }
}
```

Points de vigilance

- **Gestion de la mémoire** : Bien que Dart possède un Garbage Collector, évitez de créer des objets inutiles dans la méthode `build()` des widgets, car celle-ci est appelée fréquemment.
- **Null Safety** : Adoptez le typage strict. Évitez l'usage excessif de `!`, qui force le déballage d'une valeur potentiellement nulle et peut provoquer un crash.
- **Performance des shaders** : Avec l'adoption d'Impeller, la fluidité est accrue. Assurez-vous de cibler les versions récentes de Flutter pour bénéficier des optimisations liées à ce moteur.

Erreurs fréquentes

- **Bloquer le thread principal** : Effectuer des calculs lourds (ex: traitement d'image, parsing JSON massif) directement dans le thread UI. Utilisez `compute()` pour déléguer ces tâches à un *Isolate* (thread séparé).
- **Ignorer les avertissements du compilateur** : Les warnings de typage sont souvent des indicateurs de bugs potentiels en production.

Sources

- [Dart Language Documentation - Null Safety](#)
- [Flutter Documentation - Impeller Rendering Engine](#)
- [Flutter Documentation - Understanding the Flutter Engine](#)

1-1-3 Concepts fondamentaux : Widgets Stateless vs Stateful

Dans Flutter, tout est widget. L'interface utilisateur est une arborescence de widgets qui décrivent la configuration de l'UI. La distinction entre `StatelessWidget` et `StatefulWidget` est le fondement de la gestion de l'affichage.

1. StatelessWidget : L'affichage immuable

Un `StatelessWidget` est un widget qui ne change pas au cours de la durée de vie de l'application. Il ne dépend que des paramètres reçus lors de sa création (ses `properties`).

- **Fonctionnement** : Il est construit une seule fois. Si ses paramètres changent, le widget est détruit et reconstruit avec de nouvelles valeurs.
- **Cas d'usage** : Icônes, textes statiques, boutons de navigation simples, éléments de mise en page (padding, colonnes).

```
class SimpleLabel extends StatelessWidget {  
  final String text;  
  const SimpleLabel({super.key, required this.text});  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(text);  
  }  
}
```

2. StatefulWidget : La gestion de l'état dynamique

Un `StatefulWidget` est utilisé lorsque l'interface doit réagir à des changements de données internes (ex: saisie utilisateur, réponse API, animation).

- **Fonctionnement** : Il se compose de deux classes :
 1. La classe `StatefulWidget` (immuable).
 2. La classe `State` (mutable), qui contient la logique et les données.
- **Cycle de vie** : Lorsque les données changent, l'appel à `setState()` notifie le framework que l'état a été modifié, déclenchant une reconstruction de la méthode `build()`.

```
class CounterWidget extends StatefulWidget {  
  const CounterWidget({super.key});  
  
  @override  
  State<CounterWidget> createState() => _CounterWidgetState();  
}  
  
class _CounterWidgetState extends State<CounterWidget> {  
  int _counter = 0;  
  
  void _increment() {  
    setState(() {  
      _counter++; // Mise à jour de l'état et déclenchement du rebuild  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return ElevatedButton(onPressed: _increment, child: Text('Compteur: $_counter'));  
  }  
}
```

3. Comparatif technique

Caractéristique	StatelessWidget	StatefulWidget
État interne	Aucun	Oui (via la classe <code>State</code>)
Performance	Optimale (léger)	Plus lourd (gestion du cycle de vie)
Réactivité	Non	Oui (via <code>setState</code>)

4. Bonnes pratiques et points de vigilance

Stratégie de découpage

- **Remonter l'état** : Ne placez pas `StatefulWidget` à la racine de votre application. Gardez les `StatefulWidget` le plus bas possible dans l'arborescence pour limiter la zone de reconstruction lors d'un `setState()`.
- **Séparation des responsabilités** : Un widget doit se concentrer sur l'affichage. La logique métier (appels API, calculs complexes) doit être déportée dans des classes dédiées (ex: services, contrôleurs).

Erreurs fréquentes à éviter

- `setState` **inutile** : Appeler `setState` pour modifier une variable qui n'est pas utilisée dans la méthode `build`.
- **Logique métier dans `build`** : Ne jamais effectuer d'appels réseau ou de calculs lourds directement dans la méthode `build()`. Elle est appelée très fréquemment par le framework.
- **Oublier `dispose()`** : Pour les ressources comme les `AnimationController` ou les `TextEditingController`, il est obligatoire d'appeler `dispose()` dans la méthode éponyme du `State` pour éviter les fuites de mémoire.

Sources

- [Flutter Documentation - Introduction to Widgets](#)
- [Flutter Documentation - StatefulWidget class](#)

1-2-1 Configuration du SDK Flutter et de l'éditeur

La mise en place d'un environnement de développement robuste est la première étape pour garantir une productivité optimale. Flutter nécessite le SDK (Software Development Kit) et un éditeur configuré avec les extensions appropriées.

1. Installation du SDK Flutter

Le SDK Flutter contient les outils nécessaires pour compiler, déboguer et déployer vos applications.

Étapes d'installation

1. **Téléchargement** : Récupérez la version stable la plus récente sur le site officiel flutter.dev.
2. **Extraction** : Décompressez l'archive dans un répertoire permanent (ex: `C:\src\flutter` sur Windows ou `~/development/flutter` sur macOS/Linux).
3. **Configuration du PATH** : Ajoutez le sous-dossier `bin` du répertoire Flutter à votre variable d'environnement `PATH`. Cela permet d'exécuter la commande `flutter` depuis n'importe quel terminal.
4. **Vérification** : Exécutez la commande `flutter doctor` dans votre terminal. Cet outil analyse votre environnement et liste les dépendances manquantes (ex: outils Android, Xcode, simulateurs).

2. Choix et configuration de l'éditeur

Deux éditeurs sont principalement utilisés dans le milieu professionnel :

Visual Studio Code (Recommandé pour la légèreté)

VS Code est l'éditeur le plus utilisé pour Flutter en raison de sa rapidité et de son écosystème d'extensions.

- **Extension indispensable** : Installez l'extension officielle **"Flutter"** (qui inclut automatiquement l'extension **"Dart"**).
- **Avantages** : Démarrage rapide, intégration native du terminal, excellente gestion du *Hot Reload*.

Android Studio (Recommandé pour le débogage complexe)

Android Studio est un IDE complet basé sur IntelliJ.

- **Configuration** : Installez les plugins "Flutter" et "Dart" via le gestionnaire de plugins.
- **Avantages** : Outils de profilage mémoire et CPU très avancés, gestion intégrée des émulateurs Android, assistant de configuration pour les SDK natifs.

3. Comparatif des outils

Fonctionnalité	VS Code	Android Studio
Consommation RAM	Faible	Élevée
Vitesse d'ouverture	Très rapide	Lente
Outils de profilage	Basiques	Avancés
Configuration	Simple	Complète

4. Bonnes pratiques professionnelles

- **Utilisation de flutter doctor** : Avant chaque début de projet ou après une mise à jour du SDK, exécutez cette commande pour identifier les problèmes de configuration (ex: licences Android non acceptées).
- **Gestion des versions** : Utilisez un outil comme `fvm` (Flutter Version Management) pour gérer différentes versions de Flutter par projet. Cela évite les conflits lorsque vous travaillez sur plusieurs applications avec des contraintes de version différentes.
- **Emulateur vs Appareil physique** : Privilégiez le développement sur un appareil physique (mode développeur activé) pour tester les performances réelles et les interactions tactiles. Utilisez les émulateurs pour tester différentes tailles d'écran et versions d'OS.

5. Erreurs fréquentes à éviter

- **Installer Flutter dans un dossier système** : Évitez les dossiers comme `C:\Program Files` qui nécessitent des droits administrateur pour les mises à jour du SDK.
- **Ignorer les messages de `flutter doctor`** : Les erreurs concernant les licences Android (`flutter doctor --android-licenses`) sont fréquentes et bloquent la compilation.
- **Oublier de mettre à jour le SDK** : Flutter évolue rapidement. Utilisez `flutter upgrade` régulièrement pour bénéficier des dernières optimisations du moteur Impeller et des correctifs de sécurité.

Sources

- [Documentation officielle : Installation de Flutter](#)
- [Documentation officielle : Configuration de l'éditeur](#)
- [FVM \(Flutter Version Management\)](#)

1-2-2 Utilisation de 'flutter doctor' et gestion des émulateurs

La maîtrise de l'environnement de développement repose sur la capacité à diagnostiquer les problèmes de configuration et à tester l'application sur des cibles variées.

1. Diagnostic avec `flutter doctor`

La commande `flutter doctor` est l'outil de diagnostic principal. Elle vérifie l'installation du SDK, la présence des outils de build (Android SDK, Xcode, Chrome, etc.) et l'état des IDE.

Fonctionnement

- **Analyse** : Le framework interroge le système pour localiser les dépendances nécessaires.
- **Rapport** : Il affiche une liste de coches (✅) pour les éléments configurés et de croix (❌) pour les éléments manquants ou mal configurés.
- **Résolution** : Le rapport indique souvent la commande exacte à exécuter pour corriger une erreur (ex: acceptation des licences Android).

Exemple de flux de travail

```
# Lancer le diagnostic
```

```
flutter doctor
```

```
# Exemple de correction courante pour Android
```

```
flutter doctor --android-licenses
```

2. Gestion des émulateurs et simulateurs

Pour tester une application, vous devez disposer d'un environnement d'exécution.

- **Émulateur Android (AVD)** : Créé via Android Studio (Device Manager). Il permet de simuler différentes versions d'Android et différentes tailles d'écran (tablettes, téléphones pliables).
- **Simulateur iOS** : Disponible uniquement sur macOS via Xcode. Il simule les appareils Apple.
- **Appareils physiques** : Connectés via USB ou Wi-Fi (mode débogage activé).

Comparatif des environnements de test

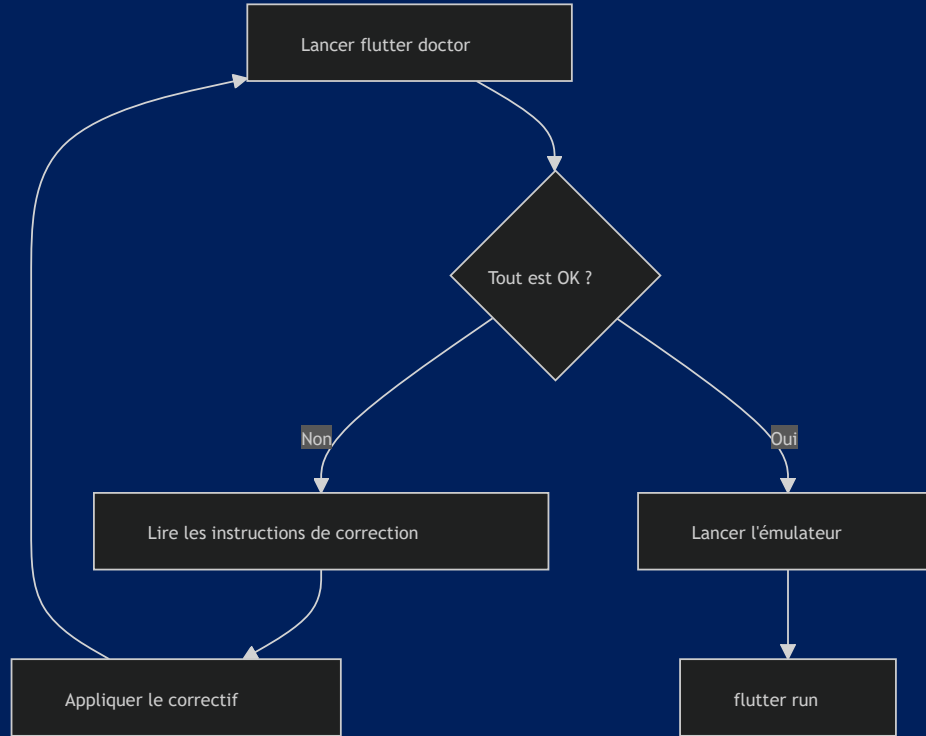
Type	Avantages	Inconvénients
Émulateur	Gratuit, multi-versions, facile à configurer	Consomme beaucoup de RAM, pas de capteurs réels
Appareil physique	Performance réelle, capteurs (GPS, Bluetooth)	Coûteux, gestion des câbles/batterie

3. Bonnes pratiques professionnelles

- **Automatisation** : Intégrez `flutter doctor` dans vos scripts de CI/CD pour vérifier que l'environnement de build est conforme avant de lancer les tests automatisés.
- **Nettoyage** : Si une application ne se compile plus après une mise à jour de dépendances, utilisez `flutter clean` suivi de `flutter pub get` avant de chercher une erreur de configuration complexe.
- **Diversité des tests** : Configurez au moins deux émulateurs : un avec une version d'OS récente et un avec une version minimale supportée par votre projet.

4. Points de vigilance et erreurs fréquentes

- **Licences Android** : L'erreur la plus fréquente après une installation est l'absence d'acceptation des licences. `flutter doctor` le détecte systématiquement.
- **Virtualisation (VT-x/AMD-V)** : Si les émulateurs Android sont extrêmement lents, vérifiez que la virtualisation est activée dans le BIOS/UEFI de votre machine.
- **Espace disque** : Les images d'émulateurs (AVD) occupent plusieurs gigaoctets. Nettoyez régulièrement les images inutilisées dans le Device Manager d'Android Studio.
- **Conflits de ports** : Si vous lancez plusieurs émulateurs, assurez-vous que votre machine dispose de suffisamment de ressources CPU, sous peine de voir le *Hot Reload* ralentir considérablement.



Sources

- [Flutter Documentation - Testing and debugging](#)
- [Flutter Documentation - Android setup](#)

2-1-1 Widgets de structure : Row, Column, Stack, Container

La composition d'interfaces dans Flutter repose sur l'imbrication de widgets. Les widgets de structure permettent d'organiser, d'aligner et de styliser les éléments visuels.

1. Organisation linéaire : Row et Column

Ces widgets permettent de disposer des éléments de manière unidimensionnelle.

- **Row (Ligne)** : Aligne les enfants horizontalement.
- **Column (Colonne)** : Aligne les enfants verticalement.

Propriétés clés

- `mainAxisAlignment` : Définit l'alignement sur l'axe principal (horizontal pour Row, vertical pour Column).
- `crossAxisAlignment` : Définit l'alignement sur l'axe secondaire (vertical pour Row, horizontal pour Column).

```
// Exemple : Une ligne avec deux éléments espacés
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Icon(Icons.person),
    Text("Profil utilisateur"),
  ],
)
```

2. Superposition : Stack

Le widget `Stack` permet de placer des widgets les uns au-dessus des autres. Il est idéal pour superposer du texte sur une image ou afficher un badge sur une icône.

- **Positionnement** : Utilisez le widget `Positioned` comme enfant direct d'une `Stack` pour définir précisément les coordonnées (top, bottom, left, right).

```
Stack(  
  children: [  
    Container(  
      width: 300,  
      height: 200,  
      color: Colors.grey,  
    ),  
  ],  
)
```

--> Suite -->

```
Positioned(
  left: 20,
  top: 30,
  child: Text(
    'Bonjour Flutter',
    style: TextStyle(fontSize: 20),
  ),
),

Positioned(
  right: 10,
  bottom: 10,
  child: ElevatedButton(
    onPressed: () {},
    child: Text('Valider'),
  ),
),
],
)
```

Ici, le texte est donc placé à (20, 30) dans le Stack, tandis que le bouton est positionné à 10 px du bas et 10 px de la droite.

3. Le couteau suisse : Container

Le `Container` est un widget polyvalent qui combine plusieurs fonctionnalités : décoration, padding, marges, taille et alignement.

- **Usage** : À utiliser pour ajouter un fond coloré, des bordures arrondies ou des espacements autour d'un widget.
- **Attention** : Un `Container` sans enfant tente de prendre toute la taille disponible, alors qu'un `Container` avec un enfant s'adapte à la taille de celui-ci.

4. Comparatif des widgets de structure

Widget	Rôle principal	Axe de disposition
Row	Alignement horizontal	Horizontal
Column	Alignement vertical	Vertical
Stack	Superposition	Z-index (profondeur)
Container	Décoration et mise en forme	N/A (boîte unique)

5. Bonnes pratiques et points de vigilance

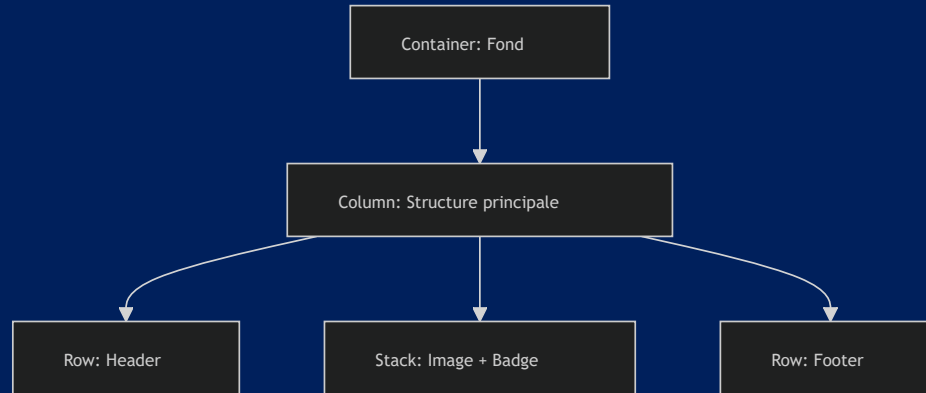
Erreurs fréquentes

- **Débordement (Overflow)** : Placer trop d'éléments dans une `Row` ou `Column` sans utiliser `Expanded` ou `Flexible` provoque une erreur de type "RenderFlex overflowed".
- **Imbrication excessive** : Créer des arbres de widgets trop profonds rend le code difficile à maintenir. Extrayez les sous-parties de votre UI dans des méthodes ou des classes de widgets dédiées.
- **Utilisation abusive de Container** : Si vous n'avez besoin que d'un padding, utilisez le widget `Padding` plutôt qu'un `Container`. Il est plus léger.

Points de vigilance

- **Flexibilité** : Utilisez `Expanded` pour forcer un enfant à occuper tout l'espace restant dans une `Row` ou `Column`.
- **Performance** : Le `Stack` est plus coûteux en ressources que `Row` ou `Column`. Utilisez-le uniquement lorsque la superposition est nécessaire.

Exemple de structure complexe



6. Bonnes pratiques

- **Design System** : Pour des interfaces cohérentes, créez vos propres widgets de structure réutilisables (ex: `AppButton`, `ProfileCard`) plutôt que de répéter des `Container` avec les mêmes propriétés de décoration.
- **Lisibilité** : Utilisez des commentaires pour délimiter les blocs de widgets complexes dans vos fichiers Dart.

Sources

- [Flutter Documentation - Layout widgets](#)
- [Flutter Documentation - Box model](#)

2-1-2 Gestion de l'espace : Padding, Margin, SizedBox, Expanded

La maîtrise de l'espacement est déterminante pour la qualité visuelle d'une interface. Flutter propose des outils spécifiques pour contrôler les dimensions et les zones vides.

1. Padding et Margin

Dans Flutter, le concept de "margin" n'existe pas en tant que propriété native de tous les widgets.

- **Padding** : Ajoute de l'espace à l'intérieur des limites d'un widget. Le widget `Padding` est la méthode standard pour appliquer cet espacement.
- **Margin** : Pour simuler une marge (espace extérieur), on utilise généralement un widget `Padding` parent ou les propriétés `margin` du widget `Container`.

```
Container(  
  margin: const EdgeInsets.all(16.0),  
  child: const Text('Contenu avec marge externe'),  
)
```

- **Container** : Widget conteneur modulable permettant la mise en forme et le positionnement.
- **Margin** : `const EdgeInsets.all(16.0)` : Crée un espace extérieur de 16 pixels tout autour du conteneur.
- **child** : `const Text(...)` : Le widget enfant placé à l'intérieur du conteneur.

```
Padding(  
  padding: const EdgeInsets.symmetric(horizontal: 12.0, vertical: 8.0),  
  child: const Text('Contenu avec espace interne'),  
)
```

- **Padding** : Widget dédié exclusivement à l'ajout d'espace interne autour de son enfant.
- **EdgeInsets.symmetric(...)** : Applique une marge interne distincte : 12 pixels sur les côtés (horizontal) et 8 pixels en haut/bas (vertical).

2. Expanded : Gestion dynamique

Le widget `Expanded` est indispensable pour occuper l'espace disponible au sein d'une `Row` ou d'une `Column`.

- **Fonctionnement** : Il force son enfant à s'étendre pour remplir l'espace restant sur l'axe principal.
- **Flex** : La propriété `flex` permet de définir des ratios (ex: un élément occupant 2/3 de l'espace et un autre 1/3).

```
Row(  
  children: [  
    Expanded(flex: 2, child: Container(color: Colors.blue)),  
    Expanded(flex: 1, child: Container(color: Colors.red)),  
  ],  
)
```

- **Row** : Aligner ses widgets enfants horizontalement sur une ligne.
- **Expanded** : Force l'enfant à occuper l'espace disponible dans la ligne.
- **flex: 2 / flex: 1** : Divise l'espace selon un ratio de 2 : 1 (66% de la largeur pour le premier conteneur, 33% pour le second).

3. Flex + SizedBox : Contrôle dimensionnel

Le `SizedBox` est un widget simple permettant de forcer des dimensions précises ou d'ajouter un espace vide entre deux éléments.

- **Espaceur** : Utilisé dans une `Row` ou `Column` pour créer un écart fixe.
- **Contrainte** : Utilisé pour forcer un enfant à avoir une largeur ou une hauteur spécifique.

```
Flex(  
  direction: Axis.horizontal,  
  children: [  
    Text('Nom'),  
    SizedBox(width: 20),  
    Text('Prénom'),  
  ],  
)
```

- **Flex** permet d'organiser des widgets horizontalement ou verticalement.
- **Axis.horizontal** → comme un `Row`
- **Axis.vertical** → comme un `Column`

4. Boutons cliquables

```
ElevatedButton(  
  onPressed: () => print('Action effectuée'),  
  child: const Text('Cliquez ici'),  
)
```

- **ElevatedButton** : Bouton matériel standard avec une légère ombre d'élévation.
- **onPressed: () => ...** : Fonction callback exécutée automatiquement lorsque l'utilisateur appuie sur le bouton.
- **child: const Text(...)** : Composant visuel affiché à l'intérieur du bouton (texte, icône, etc.).

5. Header / Footer

```
Scaffold(  
  appBar: AppBar(  
    title: Text('Mon application'),  
  ),  
  body: Center(  
    child: Text('Bienvenue'),  
  ),  
)
```

- **AppBar** représente généralement le header de l'application : titre, bouton retour, actions, etc.
- **Scaffold** fournit la structure générale de l'écran.

```
Scaffold(  
  body: Center(  
    child: Text('Accueil'),  
  ),  
  bottomNavigationBar: BottomNavigationBar(  
    items: const [  
      BottomNavigationBarItem(  
        icon: Icon(Icons.home),  
        label: 'Accueil',  
      ),  
      BottomNavigationBarItem(  
        icon: Icon(Icons.person),  
        label: 'Profil',  
      ),  
    ],  
  ),  
)
```

- **bottomNavigationBar** permet de placer une barre de navigation en bas de l'écran, souvent utilisée comme footer pour naviguer entre les principales sections.

6. Résumé

Élément	Rôle
<code>Padding</code>	Espace intérieur
<code>margin</code>	Espace extérieur
<code>Flex</code>	Organisation horizontale/verticale
<code>ElevatedButton</code>	Bouton interactif
<code>AppBar</code>	Header
<code>bottomNavigationBar</code>	Footer / navigation basse

7. Bonnes pratiques et points de vigilance

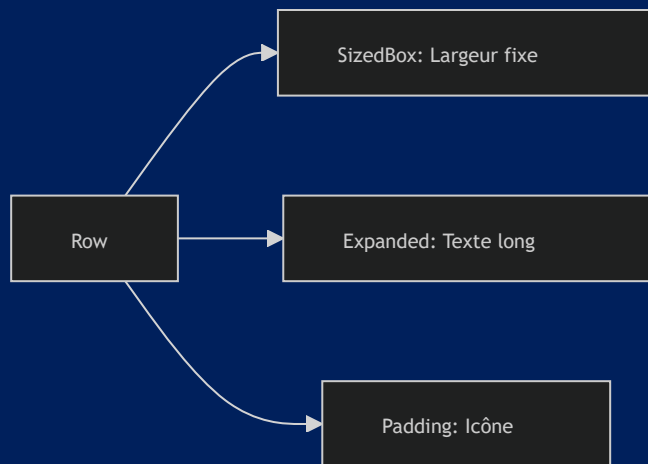
Erreurs fréquentes

- **SizedBox vs Padding** : Utiliser un `Container` avec un `padding` juste pour créer un espace vide est trop lourd. Préférez `SizedBox` pour des espaces simples ou `Padding` pour envelopper un widget existant.
- **Expanded en dehors d'un Flex** : Utiliser `Expanded` en dehors d'une `Row`, `Column` ou `Flex` provoquera une erreur à l'exécution.
- **Valeurs "en dur"** : Évitez de multiplier les valeurs numériques fixes (ex: `SizedBox(height: 20)`). Utilisez des constantes ou des variables de thème pour garantir la cohérence visuelle de l'application.

Points de vigilance

- **Flexibilité** : Si vous avez plusieurs `Expanded` dans une `Row`, ils se partageront l'espace disponible en fonction de leur valeur `flex`.
- **Débordement** : Si le contenu d'un `Expanded` est trop grand, il peut provoquer un débordement. Combinez-le avec des widgets de défilement (`SingleChildScrollView`) si nécessaire.

Exemple de mise en page professionnelle



8. Recommandations

- **Utilisation de `Spacer`** : Pour créer un espace flexible entre deux éléments dans une `Row` ou `Column`, le widget `Spacer` est une alternative plus lisible à `Expanded(child: SizedBox())`.
- **Constantes** : Définissez des espacements standards dans un fichier `constants.dart` (ex: `static const double spacing = 16.0;`) pour uniformiser le design de l'application.

Sources

- [Flutter Documentation - Padding class](#)
- [Flutter Documentation - Expanded class](#)
- [Flutter Documentation - SizedBox class](#)

3-1-1 Navigation impérative (Navigator 1.0) et routes nommées

La navigation dans Flutter repose sur le concept de pile (Stack). Le `Navigator` gère cette pile en ajoutant ou en supprimant des écrans.

1. Concepts fondamentaux : La pile de navigation

Le `Navigator` fonctionne comme une pile LIFO (*Last In, First Out*).

- **Push** : Ajoute un nouvel écran au sommet de la pile.
- **Pop** : Retire l'écran actuel pour revenir au précédent.

2. Navigation impérative (Navigator.push)

La méthode la plus directe consiste à utiliser `MaterialPageRoute`. Elle permet de définir dynamiquement l'écran de destination.

```
// Navigation vers un nouvel écran
Navigator.of(context).push(
  MaterialPageRoute(builder: (context) => const DetailsScreen()),
);

// Retour à l'écran précédent
Navigator.of(context).pop();
```

3. Routes nommées

Pour éviter de répéter les constructeurs de pages, on utilise une table de routage définie dans le `MaterialApp`.

Configuration

```
MaterialApp(  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const HomeScreen(),  
    '/details': (context) => const DetailsScreen(),  
  },  
);
```

Utilisation

```
// Navigation via le nom de la route  
Navigator.pushNamed(context, '/details');
```

4. Comparatif des approches

Approche	Avantages	Inconvénients
Push direct	Simple, typé, passage de paramètres facile	Couplage fort entre les écrans
Routes nommées	Centralisation, lisibilité	Passage de paramètres complexe

5. Gestion des paramètres

Avec les routes nommées, le passage de données complexes (objets métiers) est fastidieux. Utilisez

`ModalRoute.of(context)!.settings.arguments` pour récupérer les données transmises lors du `pushNamed`.

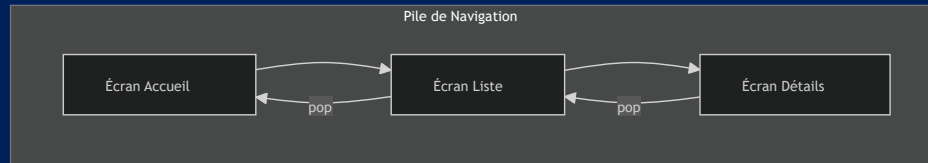
Erreurs fréquentes

- **Contexte invalide** : Appeler `Navigator` avec un `context` qui n'est pas un descendant du `Navigator` (souvent lors de l'utilisation d'un `Builder` ou d'un `async` trop long).
- **Oubli du `pop`** : Ne pas gérer le retour arrière, ce qui bloque l'utilisateur sur une page.

Points de vigilance

- **Profondeur de pile** : Une pile trop profonde consomme de la mémoire. Utilisez `pushReplacementNamed` pour remplacer l'écran actuel (ex: écran de login vers écran d'accueil) afin d'éviter que l'utilisateur ne puisse revenir en arrière sur l'écran de connexion.
- **Découplage** : Pour des applications de grande taille, la navigation impérative devient difficile à maintenir. Bien que le `Navigator 1.0` soit suffisant pour des besoins simples, envisagez des solutions basées sur des packages de routage (comme `go_router`) pour gérer les liens profonds (*deep linking*) et les états complexes.

6. Diagramme de la pile (Stack)



Sources

- [Flutter Documentation - Navigation and routing](#)
- [Flutter API - Navigator class](#)

3-1-2 Passage d'arguments lors de la transition

Le passage de données entre écrans est une nécessité pour construire des interfaces dynamiques, comme l'affichage des détails d'un produit sélectionné dans une liste.

1. Passage d'arguments via le constructeur (Navigation impérative)

C'est la méthode la plus directe et la plus sûre en termes de typage. Vous passez les données directement au constructeur de la page de destination.

```
// Écran source
Navigator.of(context).push(
  MaterialPageRoute(
    builder: (context) => DetailsScreen(productId: "123"),
  ),
);

// Écran destination
class DetailsScreen extends StatelessWidget {
  final String productId;
  const DetailsScreen({super.key, required this.productId});
  // ...
}
```

2. Passage d'arguments via les routes nommées

Lorsque vous utilisez `pushNamed`, vous devez transmettre les données via le paramètre `arguments`.

Envoi des données

```
Navigator.pushNamed(  
  context,  
  '/details',  
  arguments: {'id': '123', 'name': 'Produit A'},  
);
```

Récupération des données

Dans l'écran de destination, vous accédez aux arguments via `ModalRoute`.

```
final args = ModalRoute.of(context)!.settings.arguments as Map<String, String>;  
final productId = args['id'];
```

3. Comparatif des méthodes

Méthode	Typage	Complexité	Usage recommandé
Constructeur	Fort (Compile-time)	Faible	Navigation simple et directe
Routes nommées	Faible (Runtime)	Moyenne	Navigation centralisée, deep linking

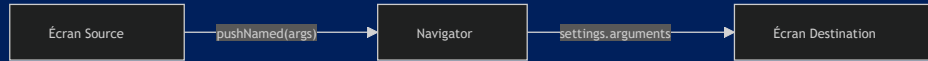
4. Bonnes pratiques

- **Utilisation de classes de données** : Au lieu de passer des `Map` ou des `dynamic`, créez des classes dédiées pour vos arguments. Cela permet de bénéficier de l'autocomplétion et de la sécurité du typage.
- **Validation** : Vérifiez toujours que les arguments reçus ne sont pas nuls avant de les utiliser, surtout avec `ModalRoute`.
- **Séparation des responsabilités** : L'écran de destination ne doit pas savoir comment les données sont récupérées. Il doit simplement recevoir les paramètres nécessaires via son constructeur.

5. Erreurs fréquentes et points de vigilance

- **Casting erroné** : Utiliser `as` sur un type incorrect lors de la récupération des arguments provoque un crash à l'exécution.
- **Arguments manquants** : Si une route nommée est appelée sans arguments alors que l'écran en attend, l'application échouera.
- **Données complexes** : Évitez de passer des objets métier très complexes ou des instances de services via la navigation. Préférez passer un identifiant (ID) et récupérer l'objet complet depuis un gestionnaire d'état ou une base de données locale.

6. Flux de transmission



Sources

- [Flutter Documentation - Passing arguments to a named route](#)
- [Flutter API - ModalRoute class](#)

3-1-3 Organisation du fichier de routes principal

La centralisation de la navigation est une étape déterminante pour la maintenabilité d'une application Flutter. Un fichier de routes bien structuré évite la dispersion de la logique de navigation dans l'ensemble du code source.

1. Centralisation des routes

Au lieu de définir les routes directement dans le `MaterialApp`, il est recommandé de les isoler dans une classe dédiée ou un fichier spécifique (ex: `app_router.dart`).

Structure recommandée

```
class AppRoutes {  
  static const String home = '/';  
  static const String details = '/details';  
  static const String settings = '/settings';  
  
  static Map<String, WidgetBuilder> get routes => {  
    home: (context) => const HomeScreen(),  
    details: (context) => const DetailsScreen(),  
    settings: (context) => const SettingsScreen(),  
  };  
}
```

2. Utilisation dans le MaterialApp

Cette approche permet de garder le fichier `main.dart` propre et lisible.

```
MaterialApp(  
  initialRoute: AppRoutes.home,  
  routes: AppRoutes.routes,  
);
```

3. Avantages de la centralisation

- **Maintenance facilitée** : Modifier le nom d'une route ou changer un écran se fait en un seul endroit.
- **Lisibilité** : Le `main.dart` ne contient plus la configuration technique des routes.
- **Réutilisation** : Les constantes de routes (`AppRoutes.home`) peuvent être utilisées partout dans l'application pour éviter les fautes de frappe lors des appels `pushNamed` .

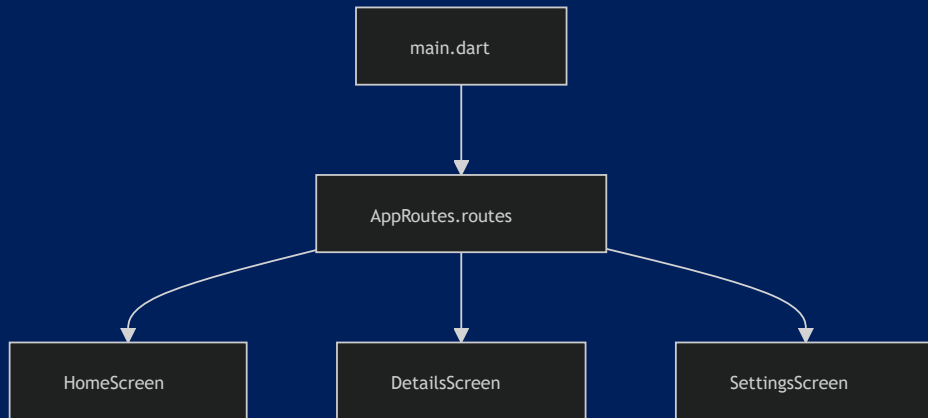
4. Bonnes pratiques

- **Utilisation de constantes** : Ne jamais utiliser de chaînes de caractères "en dur" (hardcoded) pour les noms de routes. Utilisez toujours les constantes définies dans votre classe `AppRoutes`.
- **Découpage par module** : Pour les applications de grande envergure, créez des fichiers de routes par fonctionnalité (ex: `auth_routes.dart`, `profile_routes.dart`) et fusionnez-les dans un fichier principal.
- **Gestion des routes inconnues** : Utilisez la propriété `onGenerateRoute` dans le `MaterialApp` pour gérer les cas où une route n'est pas trouvée, permettant d'afficher une page d'erreur personnalisée.

5. Erreurs fréquentes et points de vigilance

- **Importations circulaires** : Veillez à ce que votre fichier de routes n'importe pas des widgets qui, eux-mêmes, importent le fichier de routes.
- **Oubli de mise à jour** : Ajouter un écran sans l'enregistrer dans la table de routage est une source fréquente de bugs.
- **Complexité excessive** : Si votre fichier de routes dépasse quelques centaines de lignes, il est temps de le diviser par domaine fonctionnel.

6. Organisation logique



7. Recommandations

Bien que la méthode `routes` soit native et simple, elle montre ses limites pour des besoins avancés (protection de routes, redirection conditionnelle, paramètres complexes). Pour des projets professionnels, l'adoption d'un package comme `go_router` est devenue la norme. Il permet une configuration déclarative des routes, une meilleure gestion des états de navigation et une compatibilité native avec le Web (URL propres).

Sources

- [Flutter Documentation - Navigation and routing](#)
- [Go Router Package - Documentation](#)

4-1-1 Limites de setState pour les applications complexes

Le widget `StatefulWidget` et sa méthode `setState()` constituent le mécanisme de base pour mettre à jour l'interface utilisateur dans Flutter. Cependant, leur utilisation intensive dans des applications de grande envergure pose des problèmes structurels.

1. Fonctionnement de setState

`setState()` informe le framework que l'état interne d'un widget a changé, déclenchant ainsi une reconstruction de la méthode `build()` de ce widget et de ses enfants.

```
// Exemple simple : Compteur
setState(() {
  _counter++;
});
```

2. Limites dans les applications complexes

Couplage fort (Spaghetti Code)

Lorsque la logique métier est mélangée à la logique d'affichage dans le widget, le code devient difficile à tester et à maintenir.

"Prop Drilling" (Passage de paramètres en cascade)

Pour partager une donnée entre un widget parent et un petit-enfant situé plusieurs niveaux plus bas, vous devez passer cette donnée à travers chaque constructeur intermédiaire, même si ces widgets intermédiaires n'utilisent pas la donnée.

Reconstructions inutiles

`setState()` reconstruit tout le sous-arbre du widget. Dans une application complexe, cela peut entraîner des problèmes de performance si des widgets lourds sont reconstruits inutilement.

3. Comparatif : setState vs Gestion d'état externe

Caractéristique	setState	Gestion d'état (Provider/Bloc)
Portée	Locale au widget	Globale ou partagée
Logique métier	Mélangée à l'UI	Séparée de l'UI
Maintenance	Difficile à grande échelle	Facilitée par la séparation
Testabilité	Complexe (nécessite l'UI)	Simple (logique isolée)

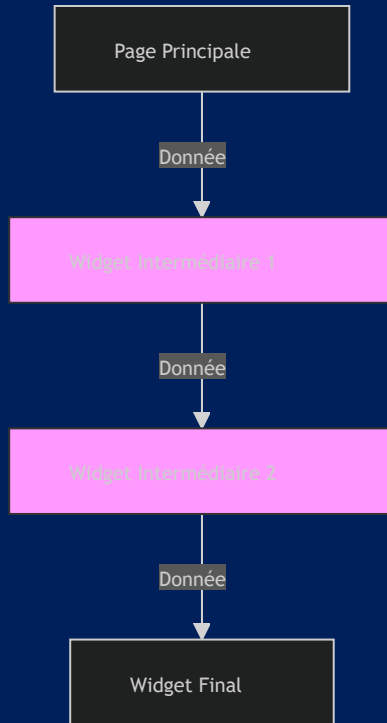
4. Cas d'usage : Quand éviter setState ?

- **Données partagées** : Lorsque plusieurs écrans ont besoin d'accéder à la même information (ex: profil utilisateur, panier d'achat).
- **Logique métier complexe** : Lorsque le traitement des données nécessite des appels API, des calculs ou des interactions avec des bases de données locales.
- **Applications multi-écrans** : Lorsque l'état doit persister lors de la navigation entre différentes routes.

5. Erreurs fréquentes et points de vigilance

- **Appel de `setState` après suppression** : Appeler `setState()` sur un widget qui a été retiré de l'arbre (ex: après une opération asynchrone) provoque une erreur. Vérifiez toujours `if (mounted)` avant.
- **Logique métier dans `build()`** : Effectuer des calculs lourds ou des appels réseau directement dans `build()` ou `setState()` ralentit l'interface.
- **Oubli de nettoyage** : Ne pas annuler les abonnements (streams, timers) dans la méthode `dispose()` crée des fuites de mémoire.

6. Illustration du problème de "Prop Drilling"



Les widgets en rose sont forcés de recevoir la donnée uniquement pour la transmettre au suivant.

7. Recommandations

- **Utilisez `setState` uniquement pour l'état éphémère** : C'est-à-dire l'état qui ne concerne qu'un seul widget (ex: état d'un champ de texte, animation locale, ouverture d'un menu déroulant).
- **Externalisez la logique métier** : Dès qu'une donnée doit être partagée ou modifiée par plusieurs composants, déplacez-la vers une classe de gestion d'état (comme `ChangeNotifier` avec `Provider`).
- **Priorisez la séparation des préoccupations** : Votre code doit être structuré de manière à ce que la logique métier puisse être testée sans instancier de widgets.

Sources

- [Flutter Documentation - Introduction to state management](#)
- [Flutter Documentation - Ephemeral vs App State](#)

4-1-2 Introduction à Provider : ChangeNotifier, ChangeNotifierProvider, Consumer

Le package `provider` est une solution recommandée par l'équipe Flutter pour la gestion d'état. Il repose sur le mécanisme d'héritage des widgets (`InheritedWidget`) tout en simplifiant considérablement son utilisation.

1. Les trois piliers de Provider

ChangeNotifier

C'est une classe simple qui étend `ChangeNotifier`. Elle contient les données de votre application et les méthodes pour les modifier. Lorsqu'une donnée change, vous appelez `notifyListeners()` pour avertir les widgets abonnés.

```
class CartModel extends ChangeNotifier {  
  int _items = 0;  
  int get items => _items;  
  
  void add() {  
    _items++;  
    notifyListeners(); // Notifie les widgets abonnés  
  }  
}
```

ChangeNotifierProvider

C'est le widget qui injecte votre instance de `ChangeNotifier` dans l'arbre des widgets. Il doit être placé au-dessus des widgets qui ont besoin d'accéder aux données.

```
ChangeNotifierProvider(  
  create: (context) => CartModel(),  
  child: MyApp(),  
);
```

Consumer

C'est un widget qui écoute les changements du `ChangeNotifier`. Il reconstruit uniquement sa propre partie de l'interface lorsque `notifyListeners()` est appelé.

```
Consumer<CartModel>(  
  builder: (context, cart, child) {  
    return Text("Articles : ${cart.items}");  
  },  
);
```

2. Fonctionnement détaillé

1. **Injection** : `ChangeNotifierProvider` crée l'instance du modèle et la rend disponible dans le contexte.
2. **Abonnement** : Le `Consumer` cherche dans le contexte le `ChangeNotifier` correspondant.
3. **Notification** : Dès que `notifyListeners()` est appelé, le `Consumer` exécute à nouveau sa fonction `builder`.

3. Comparatif : Accès aux données

Méthode	Usage	Performance
<code>Consumer</code>	Recommandé pour reconstruire une partie précise	Optimale
<code>Provider.of<T>(context)</code>	Accès direct (sans écoute)	N/A
<code>Provider.of<T>(context, listen: true)</code>	Équivalent au <code>Consumer</code>	Optimale

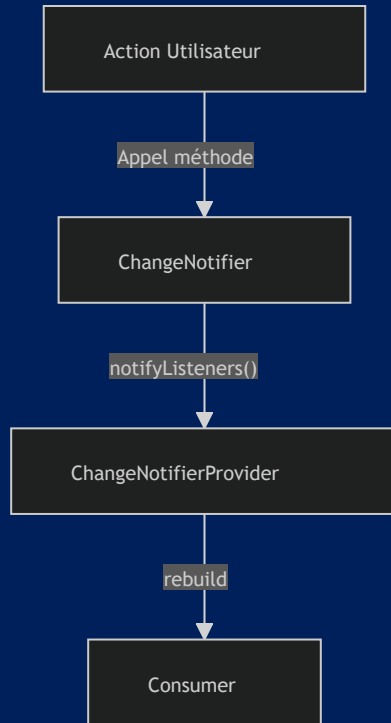
4. Bonnes pratiques

- **Placer le Provider au plus haut** : Si plusieurs écrans ont besoin de la donnée, placez le `ChangeNotifierProvider` au-dessus de `MaterialApp`.
- **Utiliser `Consumer` pour limiter les reconstructions** : Ne placez pas le `Consumer` à la racine de votre écran si seule une petite partie (ex: un compteur dans une barre d'outils) doit être mise à jour.
- **Séparation des modèles** : Créez un `ChangeNotifier` par domaine fonctionnel (ex: `AuthModel`, `ProductModel`, `CartModel`) plutôt qu'un seul modèle géant.

5. Erreurs fréquentes et points de vigilance

- **Oubli de `notifyListeners()`** : L'interface ne se mettra pas à jour si vous modifiez la donnée sans appeler cette méthode.
- **Provider introuvable** : Si vous essayez d'accéder à un `Provider` depuis un widget situé au-dessus de lui dans l'arbre, une erreur sera levée.
- **Reconstructions inutiles** : Utiliser `Provider.of<T>(context)` dans une méthode `build` complète déclenchera une reconstruction de tout le widget à chaque changement. Préférez le `Consumer` pour isoler la reconstruction.

6. Flux de données



7. Recommandation

Le package `provider` reste la référence pour les applications de taille petite à moyenne. Pour des architectures très complexes, il est souvent couplé à des patterns comme le *Repository Pattern* pour séparer la logique de récupération des données (API/Base de données) de la logique d'état.

Sources

- [Pub.dev - Provider package](#)
- [Flutter Documentation - Simple app state management](#)

4-1-3 Injection de dépendances simple

L'injection de dépendances (DI) est une technique permettant de fournir à un objet les services ou données dont il a besoin, plutôt que de les créer directement à l'intérieur de celui-ci. Avec `provider`, cette pratique devient native et très accessible.

1. Concept fondamental

Au lieu d'instancier un service (ex: `ApiService`) directement dans un widget, vous l'injectez via le `Provider`. Cela permet de découpler vos composants et de faciliter les tests unitaires en remplaçant facilement une implémentation réelle par une version "mock" (simulée).

2. Mise en œuvre avec Provider

Injection d'un service

Utilisez `Provider` (sans `ChangeNotifier`) pour injecter des objets qui ne nécessitent pas de notification de changement, comme des services API ou des dépôts de données.

```
// Injection au sommet de l'arbre
Provider<ApiService>(
  create: (context) => ApiService(),
  child: MyApp(),
);
```

Accès au service

Dans n'importe quel widget enfant, vous récupérez l'instance sans avoir à la passer via les constructeurs.

```
final apiService = Provider.of<ApiService>(context, listen: false);
apiService.fetchData();
```

3. Comparatif : Instanciation vs Injection

Approche	Dépendance	Testabilité
Instanciation directe	Forte (Couplage)	Difficile
Injection de dépendances	Faible (Découplage)	Facile (Mocking)

4. Cas d'usage

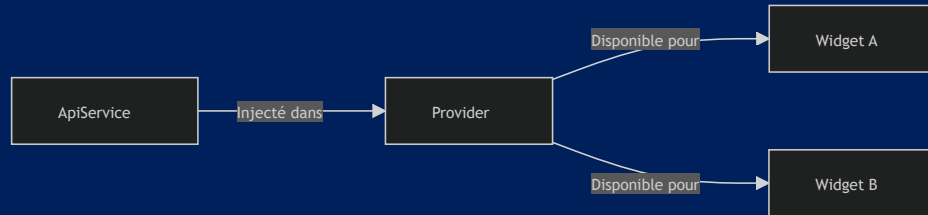
- **Services API** : Injecter un client HTTP configuré (ex: `Dio` ou `http`) pour centraliser les headers d'authentification.
- **Dépôts (Repositories)** : Injecter une couche d'abstraction entre la base de données locale et l'interface utilisateur.
- **Configuration** : Injecter des clés d'API ou des paramètres d'environnement.

5. Bonnes pratiques

- **Utilisez** `listen: false` : Pour les services qui ne changent jamais (comme un `ApiService`), utilisez toujours `listen: false` lors de la récupération. Cela évite des reconstructions inutiles du widget.
- **Interfaces (Abstract classes)** : Injectez des interfaces plutôt que des classes concrètes. Cela permet de basculer entre une implémentation `ProductionApiService` et `MockApiService` sans modifier le code des widgets.
- **MultiProvider** : Pour injecter plusieurs services, utilisez `MultiProvider` afin d'éviter l'imbrication profonde de widgets.

```
MultiProvider(  
  providers: [  
    Provider<ApiService>(create: (_) => ApiService()),  
    Provider<AuthRepository>(create: (_) => AuthRepository()),  
  ],  
  child: MyApp(),  
);
```

6. Flux de dépendances



Sources

- [Pub.dev - Provider package documentation](#)
- [Flutter Documentation - Dependency Injection](#)

5-1-1 Utilisation du package 'http'

Le package `http` est la bibliothèque standard de facto pour effectuer des requêtes HTTP dans Flutter. Il fournit des méthodes simples pour interagir avec des API REST.

1. Installation

Ajoutez la dépendance dans votre fichier `pubspec.yaml` :

```
dependencies:  
  http: ^1.2.0
```

2. Fonctionnement détaillé

Le package repose sur des méthodes asynchrones (`Future`) qui retournent un objet `Response` .

Exemple de requête GET

```
import 'package:http/http.dart' as http;

Future<void> fetchUser() async {
  final url = Uri.parse('https://api.example.com/users/1');
  final response = await http.get(url);

  if (response.statusCode == 200) {
    print(response.body);
  } else {
    throw Exception('Échec de la requête : ${response.statusCode}');
  }
}
```

Exemple de requête POST

```
final response = await http.post(
  Uri.parse('https://api.example.com/login'),
  headers: {'Content-Type': 'application/json'},
  body: jsonEncode({'username': 'admin', 'password': '123'}),
);
```

3. Comparatif des méthodes HTTP

Méthode	Usage
get	Récupérer des données (lecture seule)
post	Créer une nouvelle ressource
put	Mettre à jour une ressource existante (complète)
patch	Mettre à jour une partie d'une ressource
delete	Supprimer une ressource

4. Bonnes pratiques

- **Utilisation de `Uri.parse`** : Ne concaténez jamais des chaînes de caractères pour construire une URL. Utilisez `Uri` pour gérer correctement l'encodage des paramètres de requête.
- **Gestion des erreurs** : Vérifiez systématiquement le `statusCode`. Les codes 2xx indiquent un succès, les 4xx une erreur client, et les 5xx une erreur serveur.
- **Séparation des couches** : Ne faites jamais d'appels `http` directement dans vos widgets. Créez une classe `ApiService` ou `Repository` dédiée.
- **Timeout** : Définissez toujours un délai d'attente (timeout) pour éviter que l'application ne reste bloquée indéfiniment en cas de mauvaise connexion.

```
final response = await http.get(url).timeout(const Duration(seconds: 10));
```

5. Erreurs fréquentes et points de vigilance

- **Blocage du thread principal** : Bien que `http` soit asynchrone, le traitement du JSON (parsing) peut être coûteux. Pour de très gros volumes de données, utilisez `compute()` pour parser le JSON dans un isolate séparé.
- **Fuites de ressources** : Pour des requêtes répétées, utilisez `http.Client()` au lieu des fonctions statiques (`http.get` , `http.post`). Cela permet de réutiliser la connexion TCP (Keep-Alive).
- **Sécurité** : Ne stockez pas de jetons d'authentification (tokens) dans le code source. Utilisez des variables d'environnement ou des systèmes de stockage sécurisés (ex: `flutter_secure_storage`).

6. Flux d'une requête réseau



7. Recommandation

Le package `http` est idéal pour les besoins simples. Cependant, pour des applications professionnelles nécessitant des fonctionnalités avancées (intercepteurs pour l'authentification, gestion automatique des tokens, upload de fichiers, logs de requêtes), le package `dio` est fortement recommandé. Il offre une API plus riche et une gestion plus robuste des erreurs.

Sources

- [Pub.dev - http package](#)
- [Dart Documentation - Asynchronous programming](#)

5-1-2 Modélisation des données (Classes de modèles et fromJson)

La manipulation brute de données JSON sous forme de `Map<String, dynamic>` est source d'erreurs. La modélisation consiste à transformer ces données dynamiques en objets typés (classes Dart), garantissant ainsi la sécurité du typage et facilitant la maintenance.

1. Concept fondamental

Une classe de modèle représente la structure d'une entité métier. Elle doit être capable de :

- **Désérialiser** : Convertir un JSON (Map) en objet Dart (`fromJson`).
- **Sérialiser** : Convertir un objet Dart en JSON (`toJson`).

2. Mise en œuvre manuelle

Voici comment structurer une classe pour un utilisateur récupéré depuis une API.

```
class User {  
    final int id;  
    final String name;  
    final String email;  
  
    User({required this.id, required this.name, required this.email});  
  
    // Désérialisation  
    factory User.fromJson(Map<String, dynamic> json) {  
        return User(  
            id: json['id'] as int,  
            name: json['name'] as String,  
            email: json['email'] as String,  
        );  
    }  
  
    // Sérialisation  
    Map<String, dynamic> toJson() {  
        return {  
            'id': id,  
            'name': name,  
            'email': email,  
        };  
    }  
}
```

3. Avantages de la modélisation

- **Auto-complétion** : L'IDE propose les propriétés de l'objet, évitant les fautes de frappe sur les clés JSON.
- **Sécurité** : Les erreurs de type sont détectées lors de la compilation ou lors de la conversion, et non lors de l'utilisation dans l'interface.
- **Lisibilité** : Le code est plus explicite (`user.name` vs `user['name']`).

4. Bonnes pratiques

- **Immuabilité** : Utilisez le mot-clé `final` pour les propriétés de vos modèles afin de garantir que l'état de l'objet ne change pas après sa création.
- **Gestion des valeurs nulles** : Si une API peut renvoyer des champs manquants ou `null`, gérez-les explicitement dans le `fromJson` avec des valeurs par défaut ou des types optionnels (`String?`).
- **Validation** : Vérifiez la présence des champs obligatoires dans le `fromJson` pour éviter les exceptions `TypeError` lors de l'exécution.

5. Automatisation avec `json_serializable`

Pour les projets de taille moyenne à grande, écrire manuellement les méthodes `fromJson` et `toJson` devient fastidieux et risqué. L'utilisation du package `json_serializable` est la norme.

1. Vous annotez votre classe.
2. Un générateur de code (`build_runner`) crée automatiquement les méthodes de conversion.

```
import 'package:json_annotation/json_annotation.dart';

part 'user.g.dart'; // Fichier généré

@JsonSerializable()
class User {
  final int id;
  final String name;

  User({required this.id, required this.name});

  factory User.fromJson(Map<String, dynamic> json) => _$UserFromJson(json);
  Map<String, dynamic> toJson() => _$UserToJson(this);
}
```

6. Comparatif : Manuel vs Généré

Caractéristique	Manuel	Généré (json_serializable)
Complexité	Élevée (risque d'erreur)	Faible (automatisé)
Maintenance	Manuelle	Automatique
Performance	Excellente	Excellente
Cas d'usage	Petits projets / Prototypage	Projets professionnels

7. Erreurs fréquentes et points de vigilance

- **Types incompatibles** : Tenter de caster un `int` en `String` directement dans le `fromJson` sans conversion préalable.
- **Oubli de mise à jour** : Modifier la structure de la classe sans relancer la commande `build_runner`, ce qui entraîne des erreurs de désérialisation.
- **Noms de champs** : Si le JSON utilise des conventions différentes (ex: `snake_case` en API vs `camelCase` en Dart), utilisez `@JsonKey(name: 'user_id')` pour mapper correctement les champs.

Sources

- [Dart Documentation - JSON and serialization](#)
- [Pub.dev - json_serializable package](#)

5-1-3 Le widget FutureBuilder pour gérer les états de chargement/erreur

Le widget `FutureBuilder` est un outil puissant pour connecter l'interface utilisateur (UI) à des opérations asynchrones. Il permet de reconstruire automatiquement l'UI en fonction de l'état d'un `Future` (en attente, terminé avec succès, ou en erreur).

1. Concept fondamental

`FutureBuilder` écoute un `Future` et exécute une fonction `builder` à chaque changement d'état. Il élimine le besoin de gérer manuellement des variables d'état (`isLoading` , `errorMessage` , `data`) dans un `StatefulWidget` .

2. Fonctionnement détaillé

Le `FutureBuilder` prend deux paramètres principaux :

- `future` : L'opération asynchrone à surveiller.
- `builder` : Une fonction qui reçoit le `context` et un `AsyncSnapshot`.

L'objet `AsyncSnapshot` contient l'état actuel de la requête :

- `connectionState` : Indique si la requête est `waiting`, `active` ou `done`.
- `hasError` : Booléen indiquant si une erreur est survenue.
- `data` : Les données récupérées en cas de succès.

```
FutureBuilder<User>(  
  future: _userService.fetchUser(),  
  builder: (context, snapshot) {  
    if (snapshot.connectionState == ConnectionState.waiting) {  
      return const CircularProgressIndicator();  
    } else if (snapshot.hasError) {  
      return Text('Erreur : ${snapshot.error}');  
    } else if (snapshot.hasData) {  
      return Text('Utilisateur : ${snapshot.data!.name}');  
    }  
    return const SizedBox.shrink();  
  },  
)
```

3. États de connexion (ConnectionState)

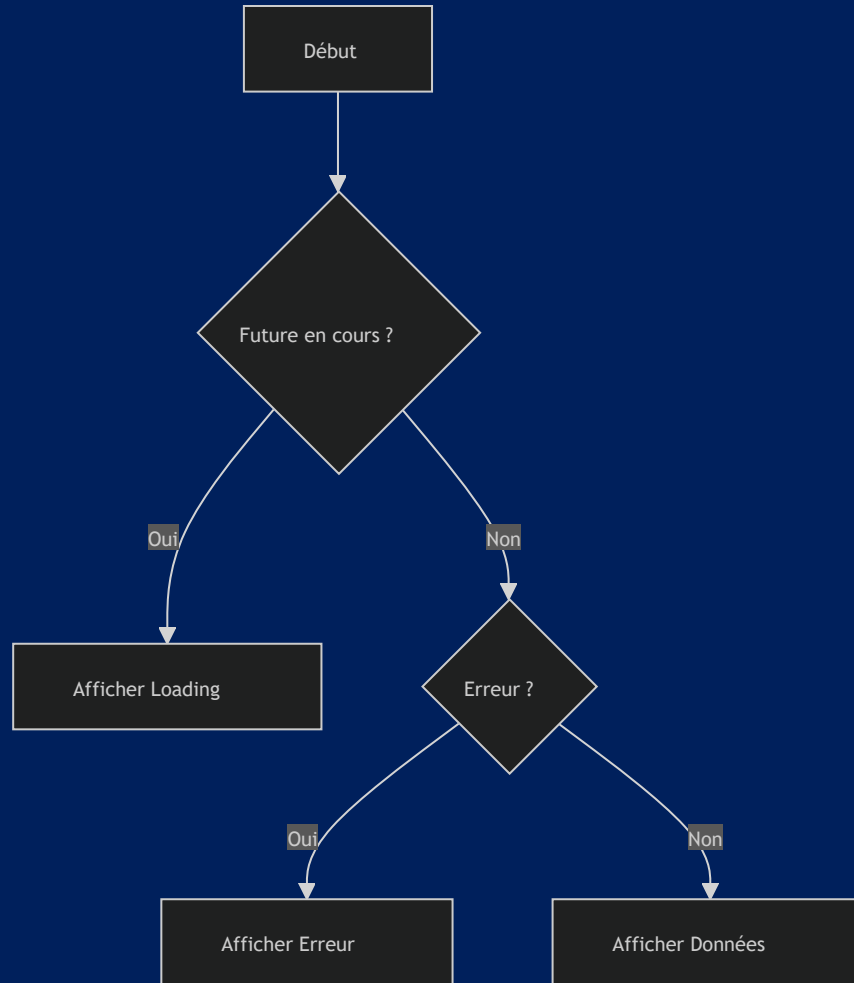
État	Signification	UI typique
<code>none</code>	Aucun futur n'est assigné	Widget vide
<code>waiting</code>	Opération en cours	Indicateur de chargement
<code>active</code>	Opération en cours (pour les Streams)	N/A
<code>done</code>	Opération terminée	Affichage des données ou erreur

4. Bonnes pratiques

- **Ne pas créer le Future dans le builder** : Le `Future` doit être créé dans `initState` ou passé via un constructeur. Si vous le créez directement dans `build`, le `Future` sera relancé à chaque reconstruction du widget, provoquant des appels API en boucle.
- **Gestion des erreurs** : Affichez toujours un message d'erreur explicite ou un bouton "Réessayer" pour améliorer l'expérience utilisateur.
- **Utilisation de `PreferredSizeWidget`** : Utilisez ce widget pour retourner un élément vide si aucun état ne correspond, plutôt que `null`.

5. Erreurs fréquentes

- **Reconstruction infinie** : Oublier de stocker le `Future` dans une variable et le recréer à chaque `build`.
- **Ignorer les états** : Ne pas gérer `snapshot.hasError` peut laisser l'utilisateur face à un écran figé en cas de problème réseau.
- **Complexité excessive** : Si la logique de gestion d'état devient trop lourde, le `FutureBuilder` devient difficile à lire. Dans ce cas, passez à une gestion d'état plus robuste (ex: `Provider` avec `ChangeNotifier`).



7. Recommandation

Le `FutureBuilder` est excellent pour des composants UI isolés et simples. Pour des écrans complexes impliquant plusieurs appels API ou une logique métier riche, préférez l'utilisation d'un `ChangeNotifier` (via `Provider`) qui offre une meilleure séparation des préoccupations et une testabilité accrue.

Sources

- [Flutter Documentation - FutureBuilder class](#)
- [Flutter Documentation - Asynchronous UI](#)

6-1-1 Widgets Form, TextFormField et GlobalKey

La saisie de données utilisateur nécessite de gérer l'état, la validation et la soumission. Flutter propose un système intégré basé sur trois composants clés : `Form` , `TextFormField` et `GlobalKey` .

1. Concepts fondamentaux

- **Form** : Un conteneur qui regroupe plusieurs champs de saisie. Il permet de valider, sauvegarder ou réinitialiser tous les champs enfants simultanément.
- **TextFormField** : Un widget de saisie de texte spécialisé qui s'intègre nativement au widget `Form` . Il possède une propriété `validator` pour définir les règles de validation.
- **GlobalKey < FormState >** : Une clé unique permettant d'accéder à l'état du formulaire (`FormState`) depuis n'importe où dans le widget, afin de déclencher les méthodes de validation ou de sauvegarde.

2. Mise en œuvre

Structure de base

```
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        validator: (value) {
          if (value == null || value.isEmpty) return 'Champ requis';
          return null;
        },
      ),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            // Le formulaire est valide
          }
        },
        child: const Text('Soumettre'),
      ),
    ],
  ),
)
```

3. Fonctionnement détaillé

1. **Enregistrement** : Chaque `TextFormField` s'enregistre automatiquement auprès du `Form` parent via la `GlobalKey` .
2. **Validation** : Lors de l'appel à `_formKey.currentState!.validate()` , le `Form` parcourt tous ses `TextFormField` enfants et exécute leurs fonctions `validator` .
3. **Feedback** : Si un validateur retourne une chaîne de caractères, le champ affiche cette erreur sous le champ de saisie. Si le validateur retourne `null` , le champ est considéré comme valide.

4. Comparatif : TextField vs TextFormField

Caractéristique	TextField	TextFormField
Intégration Form	Non	Oui
Validation native	Non	Oui (<code>validator</code>)
Sauvegarde native	Non	Oui (<code>onSaved</code>)
Usage	Recherche simple, filtre	Inscription, Login, Profil

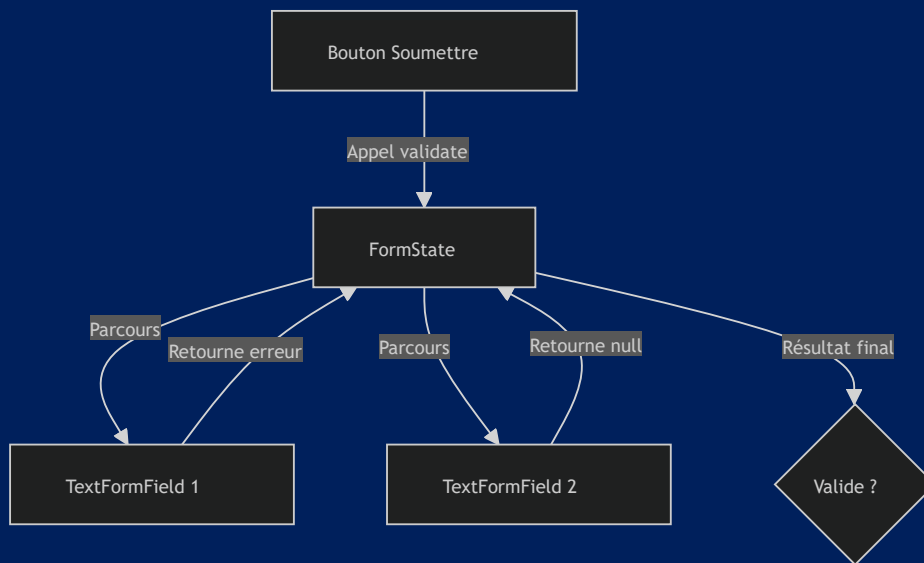
5. Bonnes pratiques

- **Gestion des données** : Utilisez `onSaved` pour transférer les valeurs saisies vers un objet métier (ex: un modèle `User`) une fois la validation réussie.
- **Feedback utilisateur** : Utilisez `autovalidateMode` sur le `Form` pour déclencher la validation dès que l'utilisateur commence à taper (`AutovalidateMode.onUserInteraction`).
- **Clavier adapté** : Configurez `keyboardType` (ex: `TextInputType.emailAddress`, `TextInputType.number`) pour améliorer l'expérience utilisateur.
- **Sécurité** : Pour les mots de passe, utilisez `obscureText: true` et `enableSuggestions: false`.

6. Erreurs fréquentes et points de vigilance

- **Oubli de la clé** : Si la `GlobalKey` n'est pas passée au widget `Form`, les méthodes `validate()` ou `save()` ne fonctionneront pas.
- **Validation bloquante** : Ne réalisez pas d'opérations asynchrones lourdes (ex: appel API) directement dans le `validator`. La validation doit être synchrone et rapide.
- **Cycle de vie** : La `GlobalKey` doit être définie comme une propriété de l'état du widget (`State`) et non recrée à chaque méthode `build`.

7. Flux de validation



Sources

- [Flutter Documentation - Building a Form with Validation](#)
- [Flutter API - Form class](#)

6-1-2 Logique de validation (Regex, longueur, champs obligatoires)

La validation des données saisies par l'utilisateur est une étape indispensable pour garantir l'intégrité des données envoyées à votre backend. Elle s'effectue principalement via la propriété `validator` du widget `TextFormField`.

1. Concepts fondamentaux

La fonction `validator` reçoit la valeur actuelle du champ sous forme de `String?`.

- Si elle retourne `null`, la validation est réussie.
- Si elle retourne une `String` (le message d'erreur), la validation échoue et le message s'affiche sous le champ.

2. Techniques de validation courantes

Champs obligatoires

La vérification la plus simple consiste à s'assurer que la chaîne n'est pas vide.

```
validator: (value) {  
  if (value == null || value.trim().isEmpty) {  
    return 'Ce champ est obligatoire';  
  }  
  return null;  
}
```

Longueur minimale et maximale

Utile pour les mots de passe ou les noms d'utilisateur.

```
validator: (value) {  
  if (value  $\neq$  null && value.length < 8) {  
    return 'Minimum 8 caractères requis';  
  }  
  return null;  
}
```

Expressions régulières (Regex)

Les Regex permettent de valider des formats complexes comme les emails ou les numéros de téléphone.

```
final emailRegex = RegExp(r'^[\w-\.\.]+\@([\w-]+\.\.)+[\w-]{2,4}$');  
  
validator: (value) {  
  if (value == null || !emailRegex.hasMatch(value)) {  
    return 'Format email invalide';  
  }  
  return null;  
}
```

3. Synthèse des cas d'usage

Type de donnée	Règle de validation
Nom / Prénom	Non vide, longueur min 2
Email	Regex standard
Mot de passe	Longueur min, présence de chiffres/caractères spéciaux
Code Postal	Regex numérique (ex: <code>^\d{5}\$</code>)

4. Bonnes pratiques

- **Centralisation** : Créez une classe utilitaire `Validators` contenant des méthodes statiques réutilisables pour éviter la duplication de code.
- **Trim** : Utilisez toujours `.trim()` sur les entrées utilisateur pour ignorer les espaces inutiles au début ou à la fin de la saisie.
- **Messages clairs** : Fournissez des messages d'erreur explicites qui indiquent à l'utilisateur comment corriger son erreur.
- **Validation côté serveur** : La validation côté client est une aide à l'ergonomie. Elle ne remplace jamais la validation côté serveur, qui est la seule garante de la sécurité des données.

5. Exemple de classe utilitaire

```
class Validators {  
    static String? email(String? value) {  
        final regex = RegExp(r'^[\w-\.]@([\w-]+\.)+[\w-]{2,4}$');  
        if (value == null || !regex.hasMatch(value)) return 'Email invalide';  
        return null;  
    }  
}  
  
// Utilisation  
TextFormField(validator: Validators.email)
```

6. Erreurs fréquentes et points de vigilance

- **Regex complexes** : Évitez les Regex trop complexes qui peuvent devenir illisibles et difficiles à maintenir. Si une règle est trop complexe, décomposez-la en plusieurs étapes de validation.
- **Validation trop stricte** : Ne bloquez pas des saisies valides par excès de zèle (ex: interdire certains caractères spéciaux dans un nom de famille).
- **Performance** : Bien que la validation soit rapide, évitez les calculs intensifs dans le `validator` car il est appelé fréquemment lors de la saisie si `autovalidateMode` est activé.

Sources

- [Dart Documentation - RegExp class](#)
- [Flutter Cookbook - Form Validation](#)

6-1-3 Gestion des contrôleurs (TextEditingController)

Le `TextEditingController` est l'objet qui permet de contrôler, lire et modifier le contenu d'un champ de texte (`TextField` ou `TextFormField`) de manière impérative.

1. Concepts fondamentaux

Contrairement à la validation qui est déclarative, le contrôleur offre un accès direct à la valeur saisie. Il permet de :

- Récupérer la valeur actuelle (`controller.text`).
- Définir une valeur par défaut ou modifier le texte par programmation.
- Écouter les changements en temps réel.

2. Mise en œuvre

Un contrôleur doit être instancié dans le `State` d'un widget et, surtout, **libéré** lors de la destruction du widget pour éviter les fuites de mémoire.

```
class MyForm extends StatefulWidget {  
  @override  
  _MyFormState createState() => _MyFormState();  
}  
  
class _MyFormState extends State<MyForm> {  
  final _controller = TextEditingController();  
  
  @override  
  void dispose() {  
    _controller.dispose(); // Libération obligatoire  
    super.dispose();  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return TextFormField(controller: _controller);  
  }  
}
```

3. Fonctionnalités avancées

Écouter les changements

Vous pouvez attacher un écouteur pour réagir à chaque frappe clavier :

```
_controller.addListener(() {  
  print('Valeur actuelle : ${_controller.text}');  
});
```

Manipulation du curseur

Le contrôleur permet de gérer la position du curseur ou de sélectionner du texte :

```
_controller.selection = TextSelection.collapsed(offset: _controller.text.length);
```

4. Comparatif : `onChanged` vs `TextEditingController`

Caractéristique	<code>onChanged</code>	<code>TextEditingController</code>
Approche	Réactive (callback)	Impérative (accès direct)
Complexité	Faible	Moyenne
Cas d'usage	Mise à jour d'état simple	Manipulation complexe, accès externe
Performance	Légère	Légère

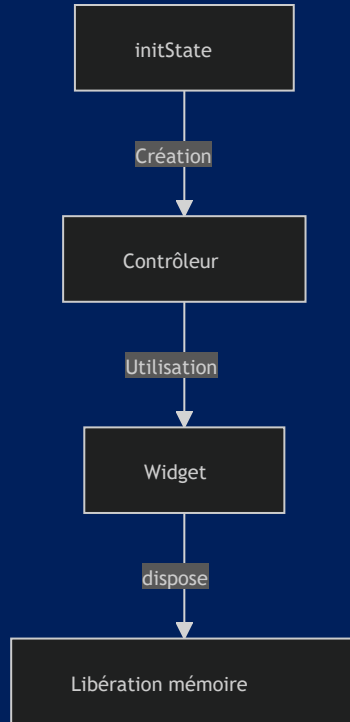
5. Bonnes pratiques

- **Dispose systématiquement** : L'oubli du `.dispose()` est une cause fréquente de fuites de mémoire dans les applications Flutter.
- **Utilisation ciblée** : N'utilisez un contrôleur que si vous avez besoin de manipuler le texte en dehors de la simple récupération lors de la soumission.
Pour un formulaire classique, `onSaved` ou `onChanged` suffisent souvent.
- **Initialisation** : Si vous devez pré-remplir un champ (ex: modification de profil), passez la valeur initiale via le constructeur du contrôleur :
`TextEditingController(text: 'Valeur initiale')`.

6. Erreurs fréquentes et points de vigilance

- **Réinitialisation du curseur** : Modifier la propriété `.text` d'un contrôleur déplace souvent le curseur au début du champ. Si vous mettez à jour le texte dynamiquement, vous devez gérer manuellement la position du curseur.
- **Conflit avec `initialValue`** : Vous ne pouvez pas utiliser `initialValue` dans un `TextFormField` si vous lui avez déjà assigné un `controller`. Le contrôleur prend le pas sur la propriété.
- **Reconstruction** : Ne créez jamais un `TextEditingController` directement dans la méthode `build`. Il sera recréé à chaque reconstruction, perdant ainsi le focus et le texte saisi.

7. Cycle de vie du contrôleur



Sources

- [Flutter API - TextEditingController class](#)
- [Flutter Documentation - Handling changes to a text field](#)

7-1-1 Utilisation du package SharedPreferences

Le package `shared_preferences` est la solution standard dans l'écosystème Flutter pour stocker de petites quantités de données persistantes sous forme de paires clé-valeur.

1. Concepts fondamentaux

`SharedPreferences` permet de sauvegarder des données simples (booléens, entiers, doubles, chaînes de caractères et listes de chaînes) sur le disque de l'appareil. Ces données persistent même après la fermeture ou le redémarrage de l'application.

2. Fonctionnement détaillé

L'accès au stockage est asynchrone. Vous devez obtenir une instance de `SharedPreferences` avant de pouvoir lire ou écrire des données.

Lecture et écriture

```
import 'package:shared_preferences/shared_preferences.dart';

// Écriture
final prefs = await SharedPreferences.getInstance();
await prefs.setInt('counter', 42);

// Lecture
final int? counter = prefs.getInt('counter');
```

3. Cas d'usage

- **Préférences utilisateur** : Thème sombre/clair, langue choisie.
- **État de session** : Mémoriser si l'utilisateur est déjà connecté (token d'authentification).
- **Configuration** : Activation/désactivation de notifications, tutoriel déjà vu.

4. Avantages et limites

Avantages	Limites
Simple à mettre en œuvre	Uniquement pour des données simples
Persistance native	Non sécurisé pour les données sensibles
Rapide pour de petits volumes	Pas de requêtes complexes (pas de SQL)

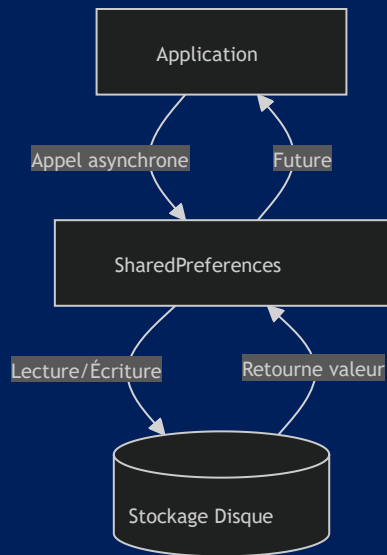
5. Bonnes pratiques

- **Injection de dépendances** : Ne pas appeler `SharedPreferences.getInstance()` partout dans le code. Initialisez-le une fois au démarrage de l'application et injectez l'instance dans vos services.
- **Clés constantes** : Définissez vos clés de stockage comme des constantes (`static const String keyUserToken = 'user_token';`) pour éviter les erreurs de frappe.
- **Données sensibles** : N'utilisez **jamais** `SharedPreferences` pour stocker des mots de passe ou des clés API privées. Utilisez des solutions sécurisées comme `flutter_secure_storage` pour ces cas.

6. Erreurs fréquentes et points de vigilance

- **Blocage de l'UI** : Bien que l'accès soit rapide, les opérations sont asynchrones. Assurez-vous de toujours utiliser `await` pour éviter des comportements imprévisibles.
- **Volume de données** : Si vous commencez à stocker des listes complexes ou des objets JSON volumineux, passez à une base de données locale comme `sqflite` ou `isar`.
- **Gestion des types** : `SharedPreferences` est typé. Tenter de lire une clé avec le mauvais type (ex: `getInt` sur une clé contenant une `String`) retournera `null` ou provoquera une erreur.

7. Flux de persistance



8. Recommendations

Pour les applications modernes, il est recommandé de créer une couche d'abstraction (un service) autour de `SharedPreferences`. Cela permet de changer facilement de technologie de stockage à l'avenir sans modifier le reste de votre code métier.

Sources

- [Pub.dev - shared_preferences package](#)
- [Flutter Documentation - Persisting data](#)

7-1-2 Stockage de fichiers simples sur le disque

Pour stocker des données plus volumineuses ou structurées (JSON, logs, images) que ce que permettent les préférences, Flutter utilise le système de fichiers local via le package `path_provider` et la bibliothèque `dart:io`.

1. Concepts fondamentaux

Le système de fichiers est organisé en répertoires accessibles par l'application :

- **Application Documents Directory** : Répertoire privé de l'application, idéal pour les données créées par l'utilisateur.
- **Temporary Directory** : Répertoire pour les fichiers cache qui peuvent être supprimés par le système à tout moment.

2. Fonctionnement détaillé

La manipulation de fichiers nécessite trois étapes :

1. **Localisation** : Trouver le chemin du répertoire avec `path_provider` .
2. **Accès** : Créer une instance de `File` avec `dart:io` .
3. **Opération** : Lire ou écrire le contenu.

```
import 'dart:io';
import 'package:path_provider/path_provider.dart';

Future<String> get _localPath async {
  final directory = await getApplicationDocumentsDirectory();
  return directory.path;
}

Future<File> get _localFile async {
  final path = await _localPath;
  return File('$path/data.txt');
}

// Écriture
Future<File> writeData(String data) async {
  final file = await _localFile;
  return file.writeAsString(data);
}

// Lecture
Future<String> readData() async {
  final file = await _localFile;
  return await file.readAsString();
}
```

3. Comparatif : SharedPreferences vs Fichiers

Caractéristique	SharedPreferences	Fichiers (dart.io)
Type de données	Clé-valeur simple	Tout type (texte, binaire)
Complexité	Très faible	Moyenne
Volume	Petit (< 1 Mo)	Illimité (selon stockage)
Structure	Plate	Hiérarchique

4. Cas d'usage

- **Export de données** : Générer un fichier CSV ou JSON pour un rapport.
- **Cache d'images** : Stocker des images téléchargées pour éviter de les recharger.
- **Logs d'erreurs** : Enregistrer des traces d'exécution pour le débogage.

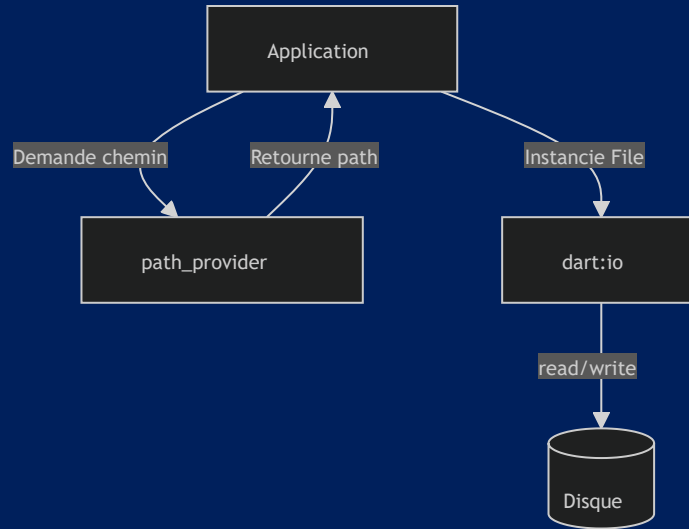
5. Bonnes pratiques

- **Gestion des erreurs** : Enveloppez toujours les opérations de lecture/écriture dans des blocs `try-catch` pour gérer les problèmes d'accès disque ou de permissions.
- **Asynchronisme** : Toutes les opérations sur le système de fichiers sont bloquantes pour le thread principal. Utilisez systématiquement les méthodes `async` pour maintenir la fluidité de l'interface.
- **Nettoyage** : Si vous utilisez le répertoire temporaire, implémentez une logique pour purger les vieux fichiers afin de ne pas saturer l'espace disque de l'utilisateur.

6. Erreurs fréquentes et points de vigilance

- **Chemins codés en dur** : N'utilisez jamais de chemins absolus (ex: `/data/user/ ...`). Utilisez toujours les méthodes de `path_provider` pour obtenir les chemins dynamiquement selon l'OS.
- **Concurrence** : Évitez d'écrire dans le même fichier depuis plusieurs endroits simultanément. Cela peut corrompre les données.
- **Permissions** : Sur Android et iOS, l'accès aux répertoires de l'application ne nécessite pas de permissions spéciales. Si vous accédez à des répertoires publics (ex: galerie photo), vous devrez configurer les permissions dans `AndroidManifest.xml` ou `Info.plist` .

7. Flux de travail avec les fichiers



Sources

- [Pub.dev - path_provider package](#)
- [Dart Documentation - dart:io File class](#)

7-1-3 Cycle de vie des données locales

La gestion du cycle de vie des données est le processus consistant à définir quand les données sont créées, mises à jour, persistées et supprimées. Une mauvaise gestion entraîne une accumulation de données inutiles (le "junk data") et des problèmes de confidentialité.

1. Concepts fondamentaux

Le cycle de vie des données locales suit généralement ces phases :

- **Initialisation** : Création ou récupération des données lors du premier lancement.
- **Persistence** : Sauvegarde des changements effectués par l'utilisateur.
- **Synchronisation** : Mise à jour avec un serveur distant (si applicable).
- **Purge** : Suppression des données obsolètes ou lors de la déconnexion.

2. Stratégies de gestion

Données éphémères (Cache)

Ces données peuvent être recréées à partir d'une source distante. Elles doivent être stockées dans le répertoire temporaire et purgées régulièrement.

Données persistantes (État utilisateur)

Ces données doivent survivre à la fermeture de l'application. Elles sont stockées dans le répertoire des documents.

3. Cycle de vie et suppression

Type de donnée	Stratégie de suppression
Cache (images, API)	Purge automatique si espace disque faible ou après X jours.
Session (Token)	Suppression lors de la déconnexion explicite.
Préférences	Conservation jusqu'à la désinstallation ou réinitialisation.
Logs	Rotation des fichiers (suppression des plus anciens).

4. Bonnes pratiques professionnelles

- **Déconnexion propre** : Lors d'une déconnexion, effacez systématiquement les données sensibles (`SharedPreferences` , bases de données locales, fichiers chiffrés).
- **Gestion de l'espace disque** : Vérifiez l'espace disponible avant d'écrire de gros fichiers.
- **Versionnage des données** : Si vous modifiez la structure de vos données (ex: ajout d'un champ dans un JSON), prévoyez une logique de migration pour éviter de corrompre les données existantes sur le téléphone de l'utilisateur.

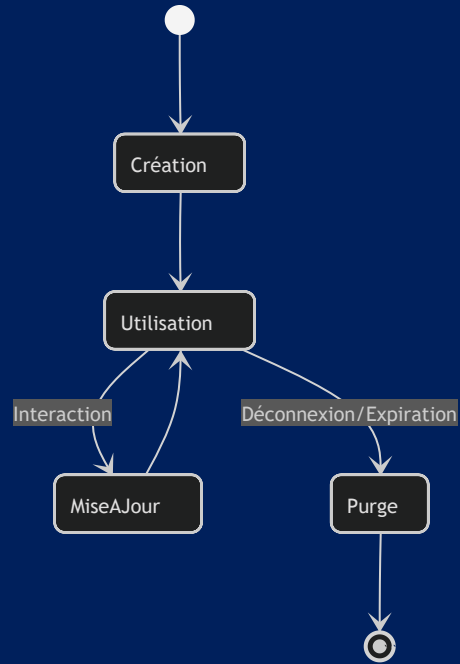
5. Erreurs fréquentes et points de vigilance

- **Données orphelines** : Oublier de supprimer les fichiers associés à un objet supprimé (ex: supprimer un profil utilisateur mais garder son avatar sur le disque).
- **Persistance excessive** : Stocker des données qui devraient être re-téléchargées. Cela augmente la taille de l'application et peut rendre les données obsolètes.
- **Non-respect de la vie privée** : Conserver des données personnelles après que l'utilisateur a supprimé son compte.

6. Exemple : Logique de déconnexion

```
Future<void> logout() async {  
    final prefs = await SharedPreferences.getInstance();  
    final cacheDir = await getTemporaryDirectory();  
  
    // 1. Effacer les préférences  
    await prefs.clear();  
  
    // 2. Supprimer les fichiers temporaires  
    if (cacheDir.existsSync()) {  
        cacheDir.deleteSync(recursive: true);  
    }  
  
    // 3. Rediriger vers l'écran de login  
}
```

7. Diagramme de cycle de vie



8. Recommendations

Pour les applications professionnelles, implémentez une politique de rétention des données. Si votre application traite des données soumises au RGPD, la capacité à supprimer toutes les données locales d'un utilisateur est une exigence légale.

Sources

- [Android Documentation - Data storage overview](#)
- [Apple Documentation - File System Programming Guide](#)

8-1-1 Configuration de Firebase Console et FlutterFire CLI

L'intégration de Firebase dans un projet Flutter est facilitée par l'outil **FlutterFire CLI**, qui automatise la configuration spécifique à chaque plateforme (Android, iOS, Web).

1. Concepts fondamentaux

- **Firebase Console** : L'interface web où vous créez votre projet, gérez les services (Auth, Firestore, etc.) et configurez les accès.
- **FlutterFire CLI** : Un outil en ligne de commande qui génère automatiquement les fichiers de configuration nécessaires (`firebase_options.dart`) et configure les fichiers natifs (`google-services.json` pour Android, `GoogleService-Info.plist` pour iOS).

2. Fonctionnement détaillé

Prérequis

1. Avoir installé Firebase CLI sur votre machine.
2. Être connecté via `firebase login`.

Processus d'installation

Le flux de travail standard consiste à exécuter la commande suivante à la racine de votre projet :

```
dart pub global activate flutterfire_cli  
flutterfire configure
```

Cette commande effectue les actions suivantes :

1. Sélection du projet Firebase dans la console.
2. Sélection des plateformes cibles (Android, iOS, macOS, Web).
3. Génération du fichier `lib/firebase_options.dart` contenant les clés API et identifiants.
4. Mise à jour des fichiers de configuration natifs.

3. Initialisation dans le code

Une fois configuré, vous devez initialiser Firebase au démarrage de l'application :

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  runApp(MyApp());  
}
```

4. Avantages et limites

Avantages	Limites
Automatisation complète	Nécessite l'installation de Node.js/Firebase CLI
Moins d'erreurs manuelles	Dépendance à la connexion internet
Gestion multi-plateforme simplifiée	Configuration initiale parfois complexe

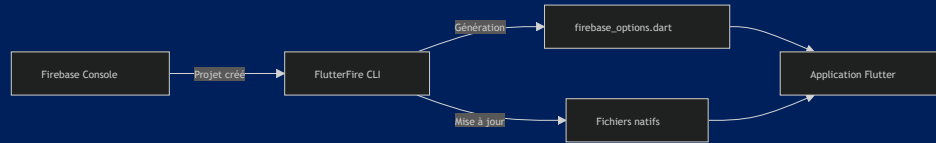
5. Bonnes pratiques professionnelles

- **Environnements multiples** : Utilisez des projets Firebase distincts pour le développement, le staging et la production. La CLI permet de configurer plusieurs environnements facilement.
- **Gestion des secrets** : Le fichier `firebase_options.dart` contient des clés publiques. Bien qu'elles ne soient pas secrètes par nature, évitez de les modifier manuellement.
- **Versionnage** : Ajoutez `firebase_options.dart` à votre gestionnaire de version (Git).

6. Erreurs fréquentes et points de vigilance

- **Oubli de `ensureInitialized`** : L'initialisation de Firebase nécessite que le binding Flutter soit prêt. Toujours appeler `WidgetsFlutterBinding.ensureInitialized()` avant `initializeApp`.
- **Conflits de configuration** : Si vous modifiez manuellement les fichiers natifs (`AndroidManifest.xml` , `Info.plist`), la CLI peut écraser vos changements. Privilégiez toujours la CLI.
- **Projets non synchronisés** : Assurez-vous que le projet sélectionné dans la CLI correspond bien à l'environnement souhaité.

7. Flux d'intégration



Sources

- [Firebase Documentation - Add Firebase to Flutter](#)
- [FlutterFire CLI Documentation](#)

8-1-2 Firebase Auth : Connexion email/mot de passe

Firebase Authentication fournit un système complet pour gérer l'identité des utilisateurs. La méthode email/mot de passe est la plus courante pour les applications nécessitant une identification classique.

1. Concepts fondamentaux

- **Auth Instance** : Le point d'entrée pour toutes les opérations d'authentification.
- **UserCredential** : L'objet retourné après une authentification réussie, contenant les informations de l'utilisateur.
- **FirebaseAuthException** : La classe d'exception spécifique pour gérer les erreurs (email invalide, mot de passe faible, utilisateur déjà existant).

2. Fonctionnement détaillé

Inscription

```
try {  
  await FirebaseAuth.instance.createUserWithEmailAndPassword(  
    email: email,  
    password: password,  
  );  
} on FirebaseAuthException catch (e) {  
  // Gestion des erreurs (ex: weak-password, email-already-in-use)  
}
```

Connexion

```
try {  
  await FirebaseAuth.instance.signInWithEmailAndPassword(  
    email: email,  
    password: password,  
  );  
} on FirebaseAuthException catch (e) {  
  // Gestion des erreurs (ex: user-not-found, wrong-password)  
}
```

3. Gestion de l'état de connexion

Utilisez `authStateChanges()` pour écouter les changements d'état de l'utilisateur en temps réel et mettre à jour l'interface (ex: rediriger vers la page d'accueil après connexion).

```
FirebaseAuth.instance.authStateChanges().listen((User? user) {  
  if (user == null) {  
    print('Utilisateur déconnecté');  
  } else {  
    print('Utilisateur connecté : ${user.uid}');  
  }  
});
```

4. Avantages et limites

Avantages	Limites
Mise en œuvre rapide	Nécessite une gestion des emails (vérification)
Sécurité gérée par Google	Risque de phishing si non couplé à la MFA
Intégration native Flutter	Pas de récupération de mot de passe sans configuration SMTP

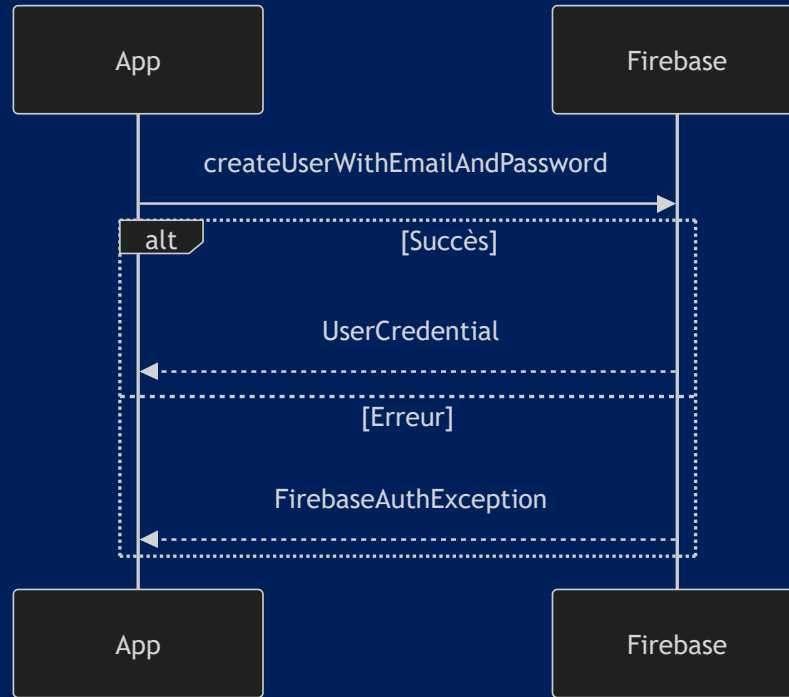
5. Bonnes pratiques

- **Validation côté client** : Validez le format de l'email et la complexité du mot de passe avant d'envoyer la requête à Firebase pour économiser des appels API.
- **Gestion des erreurs** : Traduisez les codes d'erreur Firebase (`e.code`) pour afficher des messages compréhensibles par l'utilisateur final.
- **Sécurité** : Activez la vérification d'email dans la console Firebase pour éviter les comptes frauduleux.

6. Erreurs fréquentes et points de vigilance

- **Stockage des credentials** : Ne stockez jamais l'email ou le mot de passe en clair dans `SharedPreferences` . Firebase gère automatiquement la persistance de la session de manière sécurisée.
- **Gestion du chargement** : Affichez un indicateur de chargement (`CircularProgressIndicator`) pendant l'exécution des méthodes `async` pour éviter que l'utilisateur ne clique plusieurs fois.
- **Mots de passe faibles** : Firebase rejette par défaut les mots de passe de moins de 6 caractères.

7. Flux d'authentification



Sources

- [Firebase Documentation - Authenticate with Email and Password](#)
- [FlutterFire - FirebaseAuth API Reference](#)

8-1-3 Aperçu de Cloud Firestore pour le stockage temps réel

Cloud Firestore est une base de données NoSQL orientée documents, hébergée dans le cloud, qui permet de synchroniser les données entre les clients en temps réel.

1. Concepts fondamentaux

- **Collection** : Un conteneur regroupant des documents (ex: `users`, `products`).
- **Document** : Une unité de stockage contenant des paires clé-valeur (JSON).
- **Temps réel** : La capacité de l'application à recevoir des mises à jour instantanées dès qu'une donnée change sur le serveur, sans rafraîchissement manuel.

2. Fonctionnement détaillé

Écriture de données

```
final db = FirebaseFirestore.instance;

await db.collection('tasks').add({
  'title': 'Terminer le rapport',
  'isDone': false,
  'createdAt': FieldValue.serverTimestamp(),
});
```

Lecture en temps réel

```
db.collection('tasks').snapshots().listen((snapshot) {
  for (var doc in snapshot.docs) {
    print(doc.data());
  }
});
```

3. Avantages et limites

Avantages	Limites
Synchronisation automatique	Coût lié au nombre de lectures/écritures
Mode hors-ligne natif	Requêtes complexes limitées (pas de jointures SQL)
Scalabilité automatique	Structure NoSQL exigeant une modélisation rigoureuse

4. Cas d'usage

- **Messagerie instantanée** : Affichage des nouveaux messages sans recharger la page.
- **Tableaux de bord collaboratifs** : Mise à jour des données partagées entre plusieurs utilisateurs.
- **Suivi de livraison** : Mise à jour de la position d'un colis en temps réel.

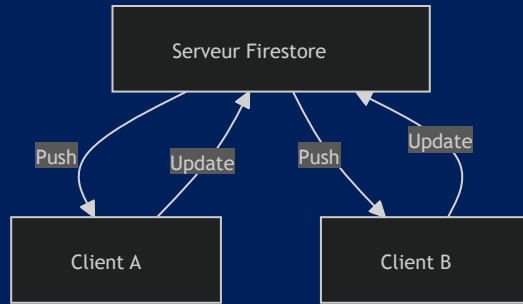
5. Bonnes pratiques

- **Modélisation** : Privilégiez la dénormalisation des données. Au lieu de faire des jointures, dupliquez les informations nécessaires dans les documents pour optimiser les lectures.
- **Sécurité** : Utilisez les **Firestore Security Rules** pour restreindre l'accès aux données. Ne faites jamais confiance au client pour valider les permissions.
- **Indexation** : Firestore crée automatiquement des index pour les requêtes simples. Pour les requêtes complexes (filtres multiples), vous devrez configurer des index composites dans la console Firebase.

6. Erreurs fréquentes et points de vigilance

- **Lecture excessive** : Écouter toute une collection (`snapshot()`) peut coûter cher en lecture si la collection est volumineuse. Utilisez des filtres (`where`) pour ne récupérer que ce qui est nécessaire.
- **Oubli de la gestion hors-ligne** : Firestore gère le cache local par défaut. Assurez-vous de comprendre comment les données sont synchronisées lors de la reconnexion.
- **Structure de données plate** : Évitez les sous-collections trop profondes qui compliquent la récupération des données.

7. Flux de synchronisation



Sources

- [Firebase Documentation - Cloud Firestore](#)
- [FlutterFire - Cloud Firestore Usage](#)

9-1-1 Animations implicites (AnimatedContainer, AnimatedOpacity)

Les animations implicites permettent de créer des transitions fluides entre deux états d'un widget sans avoir à gérer manuellement un `AnimationController`. Flutter calcule automatiquement l'interpolation entre les valeurs de début et de fin.

1. Concepts fondamentaux

- **Widget implicite** : Widget qui anime automatiquement ses propriétés lorsqu'elles changent.
- **Duration** : Temps nécessaire pour effectuer la transition.
- **Curve** : Courbe de vitesse de l'animation (ex: `Curves.easeInOut`, `Curves.bounceOut`).

2. Fonctionnement détaillé

Lorsqu'une propriété d'un widget implicite change (via un `setState`), le widget compare l'ancienne valeur à la nouvelle et anime la transition sur la durée définie.

Exemple : AnimatedContainer

```
AnimatedContainer(  
  duration: Duration(milliseconds: 500),  
  curve: Curves.easeInOut,  
  width: _isExpanded ? 200 : 100,  
  height: _isExpanded ? 200 : 100,  
  color: _isExpanded ? Colors.blue : Colors.red,  
  child: FlutterLogo(),  
)
```

Exemple : AnimatedOpacity

```
AnimatedOpacity(  
  duration: Duration(seconds: 1),  
  opacity: _isVisible ? 1.0 : 0.0,  
  child: Text("Contenu masquable"),  
)
```

3. Comparatif : Animations Implicites vs Explicites

Caractéristique	Implicites	Explicites
Complexité	Faible	Élevée
Contrôle	Automatique	Manuel (Controller)
Code	Très concis	Verbeux
Usage	Transitions d'état simples	Séquences complexes, boucles

4. Cas d'usage

- **Feedback utilisateur** : Changement de couleur d'un bouton lors d'une validation.
- **Expansion de contenu** : Agrandissement d'une carte (Card) pour afficher des détails.
- **Apparition progressive** : Affichage d'un message d'erreur ou d'un loader.

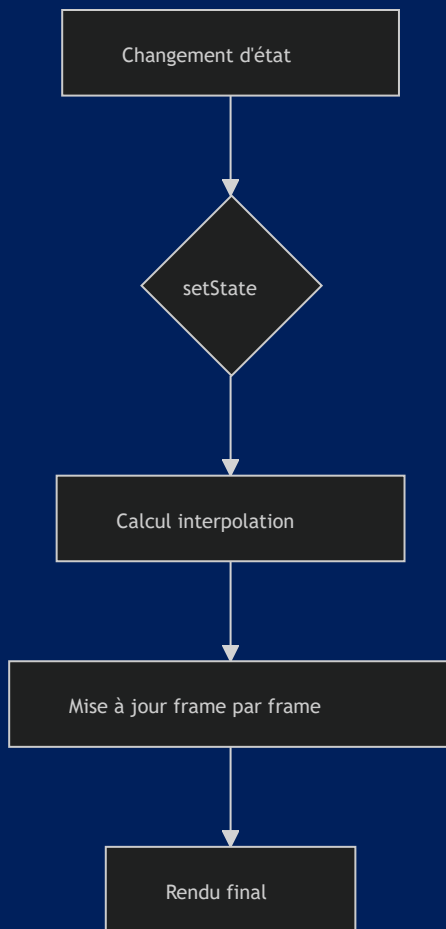
5. Bonnes pratiques

- **Cohérence** : Utilisez une durée constante pour des animations similaires dans l'application afin de maintenir une identité visuelle cohérente.
- **Courbes** : Privilégiez `Curves.easeInOut` pour la plupart des transitions, car elle offre un rendu naturel (accélération au début, décélération à la fin).
- **Performance** : Les animations implicites sont optimisées par le moteur Flutter. Évitez de déclencher des animations trop fréquentes (ex: à chaque frame) pour ne pas surcharger le processeur.

6. Erreurs fréquentes et points de vigilance

- **Oubli de `setState`** : L'animation ne se déclenchera pas si vous ne modifiez pas la valeur dans un `setState`.
- **Durées trop longues** : Une animation dépassant 500ms peut donner une impression de lenteur à l'application.
- **Conflits de propriétés** : Si vous changez plusieurs propriétés simultanément, assurez-vous que la transition globale reste lisible.

7. Flux de fonctionnement



Sources

- [Flutter Documentation - ImplicitlyAnimatedWidget](#)
- [Flutter Cookbook - Animate a widget across the screen](#)

9-1-2 Responsive Design : LayoutBuilder et MediaQuery

Le responsive design consiste à adapter l'interface utilisateur aux différentes tailles d'écran (mobiles, tablettes, ordinateurs). Flutter propose deux outils principaux pour gérer cette adaptabilité.

1. Concepts fondamentaux

- **MediaQuery** : Fournit des informations sur l'environnement global (taille de l'écran, orientation, densité de pixels).
- **LayoutBuilder** : Fournit des informations sur les contraintes (largeur/hauteur) disponibles pour le widget lui-même au sein de l'arbre des widgets.

2. Fonctionnement détaillé MediaQuery

Utilisez `MediaQuery.of(context)` pour obtenir les dimensions totales de l'appareil.

```
final size = MediaQuery.of(context).size;  
if (size.width < 600) {  
  // Affichage mobile  
} else {  
  // Affichage tablette/desktop  
}
```

LayoutBuilder

Utilisez `LayoutBuilder` pour adapter un composant en fonction de l'espace qui lui est alloué par son parent.

```
LayoutBuilder(  
  builder: (context, constraints) {  
    if (constraints.maxWidth > 800) {  
      return WideLayout();  
    } else {  
      return NarrowLayout();  
    }  
  },  
)
```

3. Comparatif : MediaQuery vs LayoutBuilder

Caractéristique	MediaQuery	LayoutBuilder
Portée	Écran entier	Widget parent uniquement
Usage	Décisions globales (ex: navigation)	Adaptabilité locale (ex: colonnes)
Performance	Déclenche un rebuild si l'écran change	Déclenche un rebuild si le parent change

4. Cas d'usage

- **Navigation** : Afficher une `BottomNavigationBar` sur mobile et un `NavigationRail` sur desktop.
- **Grilles** : Ajuster le nombre de colonnes d'une `GridView` selon la largeur disponible.
- **Typographie** : Réduire la taille des polices sur les petits écrans.

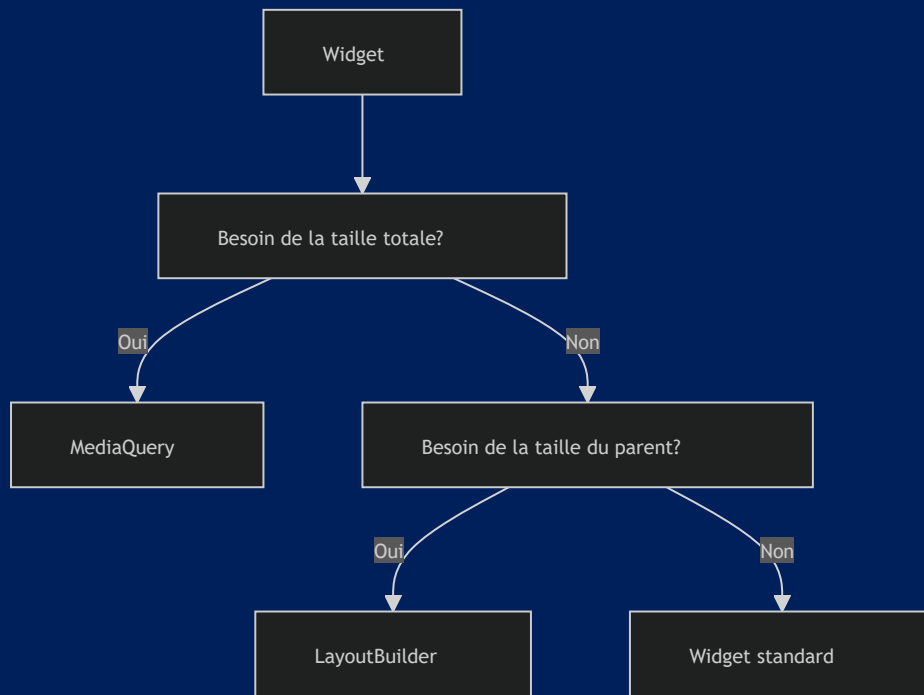
5. Bonnes pratiques

- **Approche "Mobile First"** : Concevez d'abord pour les petits écrans, puis ajoutez des éléments pour les grands écrans.
- **Points de rupture (Breakpoints)** : Définissez des constantes pour vos seuils (ex: `mobile: 600` , `tablet: 900`).
- **Utilisation ciblée** : Préférez `LayoutBuilder` pour les composants réutilisables, car il rend le widget indépendant de la taille totale de l'écran.

6. Erreurs fréquentes et points de vigilance

- **Dépendance excessive à MediaQuery** : Utiliser `MediaQuery` partout rend les widgets difficiles à tester et à réutiliser dans des parties spécifiques de l'application.
- **Ignorer l'orientation** : N'oubliez pas de vérifier `MediaQuery.of(context).orientation` pour gérer le passage du mode portrait au mode paysage.
- **Rebuilds inutiles** : `MediaQuery` peut provoquer des rebuilds fréquents. Utilisez-le uniquement là où c'est nécessaire.

7. Flux de décision responsive



8. Recommandations actuelles

Pour des projets complexes, envisagez d'utiliser des packages comme `responsive_framework` ou `flutter_adaptive_scaffold` qui standardisent la gestion des breakpoints et simplifient la création d'interfaces adaptatives.

Sources

- [Flutter Documentation - Creating responsive apps](#)
- [Flutter API - MediaQuery](#)
- [Flutter API - LayoutBuilder](#)

9-1-3 Accessibilité : Sémantique et lecteurs d'écran

L'accessibilité (a11y) garantit que les applications sont utilisables par tous, y compris les personnes souffrant de handicaps visuels, moteurs ou cognitifs. Flutter fournit une couche sémantique pour communiquer avec les technologies d'assistance (TalkBack sur Android, VoiceOver sur iOS).

1. Concepts fondamentaux

- **Arbre sémantique** : Une représentation simplifiée de l'interface que les lecteurs d'écran utilisent pour décrire le contenu à l'utilisateur.
- **Semantics Widget** : Le composant principal permettant d'annoter ou de masquer des éléments de l'interface.
- **Lecteur d'écran** : Logiciel qui lit à haute voix le contenu textuel et les actions possibles sur l'écran.

2. Fonctionnement détaillé

Flutter génère automatiquement une sémantique de base pour les widgets standards (boutons, champs de texte). Pour les composants personnalisés, vous devez intervenir manuellement.

Annotation simple

```
Semantics(  
  label: 'Valider le formulaire',  
  button: true,  
  child: IconButton(  
    icon: Icon(Icons.check),  
    onPressed: () {},  
  ),  
)
```

Masquage d'éléments décoratifs

Pour éviter que le lecteur d'écran ne lise des éléments purement visuels (ex: une icône décorative), utilisez `excludeSemantics` :

```
Semantics(  
  excludeSemantics: true,  
  child: Icon(Icons.star),  
)
```

3. Bonnes pratiques

- **Contraste des couleurs** : Respectez les ratios de contraste définis par les normes WCAG (Web Content Accessibility Guidelines).
- **Cibles tactiles** : Assurez-vous que les zones cliquables mesurent au moins 48x48 pixels pour faciliter la navigation.
- **Hiérarchie** : Utilisez des titres et des libellés clairs pour permettre aux utilisateurs de naviguer rapidement dans l'application.
- **Tests** : Testez systématiquement votre application avec TalkBack (Android) ou VoiceOver (iOS) activé.

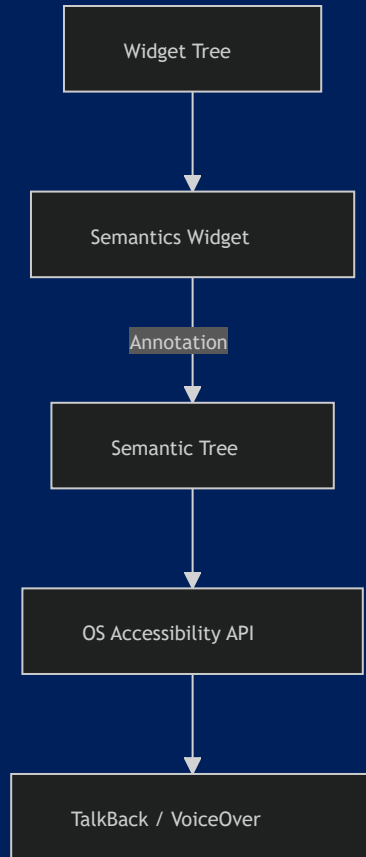
4. Erreurs fréquentes et points de vigilance

- **Libellés vagues** : Évitez les libellés comme "Bouton" ou "Image". Préférez "Valider la commande" ou "Photo du profil utilisateur".
- **Ordre de lecture** : L'ordre de lecture par défaut suit l'ordre des widgets dans l'arbre. Si l'ordre visuel diffère, utilisez `Semantics` pour réorganiser la lecture.
- **Oubli des états** : Si un bouton est désactivé, assurez-vous que le lecteur d'écran l'annonce explicitement.

5. Comparatif : Widgets standards vs Personnalisés

Élément	Sémantique automatique	Action requise
<code>ElevatedButton</code>	Oui	Aucune
<code>TextField</code>	Oui	Ajouter un <code>hintText</code>
<code>Container</code> (cliquable)	Non	Envelopper dans <code>Semantics</code>
Icône décorative	Non	<code>excludeSemantics: true</code>

6. Flux de traitement sémantique



7. Recommandations actuelles

Utilisez l'outil **Semantics Debugger** de Flutter (accessible via `showSemanticsDebugger: true` dans `MaterialApp`) pour visualiser comment votre application est interprétée par les lecteurs d'écran. Cela permet d'identifier immédiatement les zones manquantes ou mal étiquetées.

Sources

- [Flutter Documentation - Accessibility](#)
- [W3C - Web Content Accessibility Guidelines \(WCAG\)](#)

10-1-1 Architecture globale du projet final : Event Planner App

Pour le projet "Event Planner App", une architecture robuste est nécessaire pour séparer la logique métier, la gestion des données et l'interface utilisateur. Nous adopterons une structure basée sur le pattern **Layered Architecture** (Architecture en couches).

1. Concepts fondamentaux

- **Couche Présentation (UI)** : Widgets Flutter responsables de l'affichage et de l'interaction utilisateur.
- **Couche Domaine (Logique)** : Modèles de données et classes de gestion (ex: `EventService`) qui traitent les règles métier.
- **Couche Données (Data)** : Interaction avec les services externes (Firebase Auth, Firestore).

2. Structure de dossiers recommandée

Une organisation claire facilite la maintenance et la montée en charge du projet.

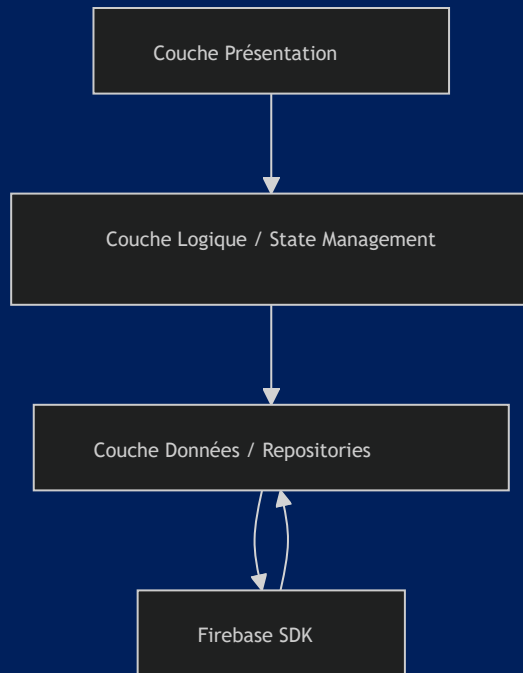
```
lib/  
├── core/           # Constantes, thèmes, utilitaires  
├── data/           # Modèles de données, repositories  
├── logic/          # Gestion d'état (ex: Provider, Bloc)  
├── presentation/   # Écrans (screens) et widgets réutilisables  
└── main.dart       # Point d'entrée
```

3. Fonctionnement détaillé

L'application suit un flux unidirectionnel :

1. **UI** : L'utilisateur interagit avec un widget (ex: bouton "Créer événement").
2. **Logic** : Le gestionnaire d'état intercepte l'action et appelle un service.
3. **Data** : Le service communique avec Firebase (Firestore/Auth).
4. **UI** : L'état est mis à jour, provoquant un rafraîchissement automatique de l'interface.

4. Diagramme d'architecture



5. Bonnes pratiques

- **Injection de dépendances** : Utilisez des packages comme `get_it` pour injecter vos services et éviter de passer des instances manuellement dans les constructeurs.
- **Modèles immuables** : Utilisez des classes `data` avec `freezed` ou `equatable` pour garantir la cohérence des données.
- **Gestion des erreurs** : Centralisez la gestion des exceptions Firebase dans la couche `data` pour renvoyer des messages d'erreur exploitables par l'UI.

1. Injection de dépendances avec `get_it`

L'injection de dépendances (DI) découple la création d'un service de son utilisation. `get_it` agit comme un **Service Locator** global et performant.

Avantages

- **Testabilité** : remplace facilement un service réel par un *mock* dans les tests.
- **Maintenabilité** : modifie le constructeur d'un service sans toucher à l'UI.

Exemple de code

Configuration du conteneur (`injection.dart`) :

```
import 'package:get_it/get_it.dart';

final sl = GetIt.instance; // sl = Service Locator

void setupLocator() {
  // LazySingleton : instancié uniquement lors du premier appel
  sl.registerLazySingleton<AuthRepository>(() => AuthRepositoryImpl());

  // Factory : une nouvelle instance est créée à chaque appel
  sl.registerFactory(() => AuthBloc(authRepository: sl<AuthRepository>()));
}
```

Initialisation et utilisation dans Flutter :

```
void main() {  
  setupLocator();  
  runApp(const MyApp());  
}  
  
// Dans un widget ou un controller :  
class ProfileScreen extends StatelessWidget {  
  const ProfileScreen({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    // Récupération directe sans passer l'instance par le constructeur  
    final authRepo = sl<AuthRepository>();  
  
    return ElevatedButton(  
      onPressed: () => authRepo.signOut(),  
      child: const Text('Déconnexion'),  
    );  
  }  
}
```

2. Modèles immuables avec `freezed`

`freezed` génère automatiquement les méthodes `copyWith`, `toString`, `operator ==` et la sérialisation JSON.

Avantages

- **Évite les effets de bord** : les données ne sont pas altérées par inadvertance.
- **Gestion d'état optimale** : des outils comme Flutter BLoC ou Riverpod dépendent de l'égalité d'instance (`==`) pour déclencher le rafraîchissement de l'UI.

Définition du modèle (`user_model.dart`) :

```
import 'package:freezed_annotation/freezed_annotation.dart';

part 'user_model.freezed.dart';
part 'user_model.g.dart';

@freezed
class UserModel with _$UserModel {
  const factory UserModel({
    required String id,
    required String email,
    required String name,
    @Default(false) bool isPremium,
  }) = _UserModel;

  factory UserModel.fromJson(Map<String, dynamic> json) =>
    _$UserModelFromJson(json);
}
```


3. Centralisation de la gestion des erreurs (Firebase)

Ne laissez jamais des exceptions brutes Firebase (`FirebaseException`) remonter jusqu'à la couche de présentation. Interceptez-les dans la couche **Data**, traduisez-les en échecs métiers (*Failures*), puis affichez un message propre dans l'UI.

Avantages

- **Expérience utilisateur (UX)** : remplace des codes d'erreur cryptiques (`auth/user-not-found`) par des messages compréhensibles.
- **Sécurité et robustesse** : centralise le comportement lors des pannes réseau ou d'API.

Définition des erreurs métiers (`failures.dart`) :

```
abstract class Failure {  
  final String message;  
  const Failure(this.message);  
}  
  
class AuthFailure extends Failure {  
  const AuthFailure(super.message);  
}  
  
class NetworkFailure extends Failure {  
  const NetworkFailure() : super('Vérifiez votre connexion internet.');
```

Implementation dans la couche Data (auth_repository_impl.dart) :

```
import 'package:firebase_auth/firebase_auth.dart';

class AuthRepositoryImpl implements AuthRepository {
  final FirebaseAuth _firebaseAuth = FirebaseAuth.instance;

  @override
  Future<void> signInWithEmail(String email, String password) async {
    try {
      await _firebaseAuth.signInWithEmailAndPassword(
        email: email,
        password: password,
      );
    } on FirebaseAuthException catch (e) {
      // Traduction des codes Firebase en exceptions/failures métiers
      switch (e.code) {
        case 'user-not-found':
        case 'wrong-password':
        case 'invalid-credential':
          throw const AuthFailure('Identifiants incorrects.');
```

Gestion dans la couche de présentation (UI) :

```
Future<void> handleLogin(BuildContext context, String email, String password) async {  
  try {  
    await sl<AuthRepository>().signInWithEmail(email, password);  
    // Redirection si succès ...  
  } on Failure catch (failure) {  
    // Affiche uniquement des messages compréhensibles pour l'utilisateur  
    ScaffoldMessenger.of(context).showSnackBar(  
      SnackBar(content: Text(failure.message)),  
    );  
  }  
}
```

6. Points de vigilance

- **Couplage** : Évitez d'importer `firebase_auth` directement dans vos widgets. Passez par une classe de service intermédiaire pour faciliter les tests unitaires.
- **Scalabilité** : Ne surchargez pas le fichier `main.dart`. Découpez vos configurations (Firebase, routes, thèmes) dans des fichiers dédiés.
- **Sécurité** : Vérifiez que vos règles Firestore sont configurées pour restreindre l'accès aux données aux seuls utilisateurs authentifiés avant de finaliser le déploiement.

7. Comparatif des approches de gestion d'état

Approche	Complexité	Cas d'usage
Provider	Faible	Projets simples, apprentissage
Riverpod	Moyenne	Projets robustes, testabilité élevée
Bloc	Élevée	Projets complexes, logique métier lourde

8. Recommandations pour le projet

Pour l'Event Planner App, **Riverpod** est recommandé pour sa flexibilité et sa capacité à gérer facilement l'état de l'authentification Firebase combiné aux données Firestore. Assurez-vous de créer un `Repository` pour chaque entité (ex: `EventRepository`, `UserRepository`) afin d'abstraire les appels API.

9. Riverpod

Riverpod est une réécriture complète de `Provider`. Il résout les limites historiques de Provider en éliminant complètement la dépendance au `BuildContext` pour lire des états, en rendant la détection des erreurs à la compilation plutôt qu'à l'exécution, et en gérant nativement l'asynchronisme.

1. Fondations : Installer et englober l'application

Active la génération de code avec `riverpod_generator` (l'approche recommandée officiellement) et englobe l'application dans un `ProviderScope`.

Dépendances (`pubspec.yaml`) :

```
dependencies:  
  flutter:  
    sdk: flutter  
  flutter_riverpod: ^2.5.1  
  riverpod_annotation: ^2.3.5  
  
dev_dependencies:  
  build_runner: ^2.4.9  
  riverpod_generator: ^2.4.0
```

Point d'entrée (`main.dart`) :

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

void main() {
  runApp(
    // Indispensable pour stocker l'état de tous les providers
    const ProviderScope(
      child: MyApp(),
    ),
  );
}
```

2. Typologie des Providers avec Code Generation

L'annotation `@riverpod` choisit automatiquement le bon provider sous-jacent selon la signature de la fonction ou classe.

A. Read-Only / Invariant (Équivalent de `Provider`)

Pour exposer un service ou un objet calculé non modifiable.

```
import 'package:riverpod_annotation/riverpod_annotation.dart';

part 'services.g.dart';

@riverpod
ApiClient apiClient(ApiClientRef ref) {
  return ApiClient(baseUrl: 'https://api.example.com');
}
```

B. Asynchrone (Équivalent de `FutureProvider` / `StreamProvider`)

Parfait pour charger des données réseau. Retourne automatiquement un `AsyncValue<T>` (*loading, data, error*).

```
part 'user_provider.g.dart';

@riverpod
Future<User> fetchUserProfile(FetchUserProfileRef ref, String userId) async {
  final api = ref.watch(apiClientProvider);
  return api.getUser(userId);
}
```

C. État Mutable (Notifier / AsyncNotifier)

Pour encapsuler une logique métier complète qui modifie l'état.

```
part 'counter_notifier.g.dart';

@riverpod
class CounterNotifier extends _$CounterNotifier {
  @override
  int build() => 0; // État initial

  void increment() => state++;
  void decrement() => state--;
}
```

3. Consommer un Provider dans l'UI (ConsumerWidget)

Remplace `StatelessWidget` par `ConsumerWidget` pour accéder au paramètre `WidgetRef ref`.

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

class CounterScreen extends ConsumerWidget {
  const CounterScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // 1. ref.watch : Écoute et reconstruit le widget à chaque changement
    final count = ref.watch(counterNotifierProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Riverpod Counter')),
      body: Center(
        child: Text('Compteur : $count', style: const TextStyle(fontSize: 24)),
      ),
      floatingActionButton: FloatingActionButton(
        // 2. ref.read : Lit une seule fois sans écouter (idéal dans les callbacks)
        onPressed: () => ref.read(counterNotifierProvider.notifier).increment(),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

4. Gérer l'Asynchronisme avec AsyncValue et le Pattern Matching

L'un des plus grands atouts de Riverpod est la manipulation sécurisée des données asynchrones via `.when()`.

```
class UserProfileScreen extends ConsumerWidget {
  final String userId;
  const UserProfileScreen({super.key, required this.userId});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // Appel du family provider avec le paramètre userId
    final asyncUser = ref.watch(fetchUserProfileProvider(userId));

    return Scaffold(
      body: asyncUser.when(
        data: (user) => Text('Bonjour ${user.name}'),
        loading: () => const CircularProgressIndicator(),
        error: (err, stack) => Text('Erreur : $err'),
      ),
    );
  }
}
```

5. Interaction entre Riverpod et GetIt / Freezed

Dans une architecture d'entreprise, Riverpod remplace avantageusement **BLoC** et **GetIt** à la fois :

- **Remplacement de GetIt** : Les dépendances (repositories, clients HTTP) sont injectées sous forme de `@riverpod` de lecture seule.
- **Remplacement de BLoC** : Les `AsyncNotifier` ou `Notifier` gèrent l'état et la logique métier.
- **Combinaison avec Freezed** : L'état d'un Notifier peut être un objet immuable `Freezed` .

Exemple d'intégration complète (Notifier + State Freezed) :

```
// 1. État Immuable
@freezed
class ProfileState with _$ProfileState {
  const factory ProfileState({
    required String name,
    @Default(false) bool isSubscribed,
  }) = _ProfileState;
}

// 2. Notifier
@riverpod
class ProfileController extends _$ProfileController {
  @override
  ProfileState build() {
    return const ProfileState(name: 'Alex');
  }

  void toggleSubscription() {
    // Immuabilité préservée grâce à copyWith
    state = state.copyWith(isSubscribed: !state.isSubscribed);
  }
}
```

Sources

- [Flutter Architecture Samples](#)
- [Riverpod Documentation](#)

10-1-2 Nettoyage du code et revue par les pairs (ou IA)

La phase de finalisation d'un projet consiste à transformer un code fonctionnel en un code maintenable, lisible et performant. Cette étape repose sur le nettoyage technique et la validation par un regard extérieur.

1. Concepts fondamentaux

- **Refactoring** : Processus de modification de la structure interne du code sans changer son comportement externe.
- **Code Review** : Examen systématique du code source par un tiers pour identifier des erreurs, des vulnérabilités ou des opportunités d'amélioration.
- **Lint** : Outil d'analyse statique qui vérifie le respect des conventions de nommage et de style (ex: `flutter_lints`).

2. Processus de nettoyage

Le nettoyage doit suivre une approche méthodique :

1. **Suppression du code mort** : Éliminer les imports inutilisés, les variables non déclarées et les fonctions commentées.
2. **Standardisation** : Appliquer le formatage automatique (`dart format .`).
3. **Simplification** : Extraire les widgets complexes en petits composants réutilisables.
4. **Nommage** : Vérifier que les noms de variables et de fonctions reflètent clairement leur rôle.

3. Revue par les pairs vs IA

Méthode	Avantages	Limites
Revue par les pairs	Partage de connaissances, contexte métier	Chronophage, dépend de la disponibilité
Revue par IA	Immédiate, disponible 24/7, exhaustive	Manque de contexte métier, hallucinations

4. Bonnes pratiques de revue

- **Checklist de revue :**
 - Le code respecte-t-il les principes SOLID ?
 - La gestion des erreurs est-elle robuste ?
 - Les accès aux données sont-ils sécurisés ?
 - La documentation (commentaires) est-elle utile et non redondante ?
- **Utilisation de l'IA :** Utilisez l'IA pour suggérer des optimisations de performance ou pour expliquer des parties complexes du code, mais validez toujours les suggestions manuellement.

5. Erreurs fréquentes à éviter

- **Revue superficielle** : Se concentrer uniquement sur le style (espaces, virgules) au lieu de la logique métier.
- **Refactoring massif sans tests** : Modifier une architecture complexe sans avoir de tests unitaires pour valider la non-régression.
- **Ignorer les warnings du Linter** : Les avertissements du compilateur indiquent souvent des problèmes de performance ou de sécurité potentiels.

7. Points de vigilance

- **Sécurité** : Vérifiez qu'aucune clé API ou donnée sensible n'est codée en dur dans le dépôt. Utilisez des fichiers `.env`.
- **Performance** : Analysez les rebuilds inutiles dans vos widgets avec l'outil *Flutter DevTools*.
- **Accessibilité** : Assurez-vous que les modifications de nettoyage n'ont pas altéré la sémantique des composants.

8. Recommandations actuelles

Intégrez l'analyse statique dans votre pipeline CI/CD. Configurez le fichier `analysis_options.yaml` avec des règles strictes pour garantir une qualité de code constante dès le développement. Pour la revue par IA, fournissez un contexte clair (ex: "Voici mon service de gestion d'événements, peux-tu vérifier les fuites de mémoire potentielles ?").

Sources

- [Dart Style Guide](#)
- [Flutter DevTools Documentation](#)

10-1-3 Introduction aux tests unitaires et publication

La phase finale d'un projet consiste à garantir sa stabilité via les tests et à préparer sa distribution sur les stores.

1. Concepts fondamentaux

- **Test unitaire** : Vérification du comportement d'une fonction, méthode ou classe isolée.
- **Mocking** : Simulation de dépendances externes (API, base de données) pour tester une logique sans effets de bord.
- **Publication** : Processus de préparation, signature et déploiement de l'application sur le Google Play Store ou l'Apple App Store.

2. Tests unitaires avec test

Flutter utilise le package `test` pour valider la logique métier.

Exemple de test

```
void main() {  
  test('Le calcul du prix total doit inclure la TVA', () {  
    final service = EventService();  
    final total = service.calculatePrice(100, 0.2);  
    expect(total, 120.0);  
  });  
}
```

3. Stratégie de publication

La publication nécessite plusieurs étapes techniques :

1. **Configuration** : Mise à jour du `pubspec.yaml` (version, nom, icône).
2. **Sécurisation** : Utilisation de fichiers `.env` pour les clés API.
3. **Build** : Génération des bundles (`flutter build appbundle` pour Android, `flutter build ipa` pour iOS).
4. **Signature** : Utilisation de certificats de signature (Keystore pour Android, Provisioning Profiles pour iOS).

4. Bonnes pratiques

- **Taux de couverture** : Visez une couverture de code significative sur la logique métier (services, modèles).
- **Automatisation** : Utilisez des outils de CI/CD (GitHub Actions, Codemagic) pour lancer les tests automatiquement à chaque `push`.
- **Gestion des versions** : Respectez le versioning sémantique (Major.Minor.Patch).

5. Erreurs fréquentes à éviter

- **Tester l'implémentation** : Testez le résultat attendu, pas la manière dont la fonction est écrite.
- **Oublier les cas limites** : Ne testez pas seulement les cas nominaux, testez aussi les entrées nulles ou les erreurs réseau.
- **Publication en mode debug** : N'oubliez jamais de compiler en mode `--release`.

8. Points de vigilance

- **Secrets** : Ne jamais inclure de fichiers contenant des clés privées dans le contrôle de version (Git).
- **Icônes et Assets** : Vérifiez la conformité des icônes avec les guidelines des stores.
- **Permissions** : Vérifiez que les permissions demandées dans `AndroidManifest.xml` et `Info.plist` sont strictement nécessaires au fonctionnement de l'application.

9. Recommandations actuelles

Pour la publication, privilégiez des outils comme **Codemagic** qui automatisent la signature et la soumission sur les stores. Pour les tests, utilisez le package `mocktail` pour simplifier la création de mocks de vos services Firebase.

Sources

- [Flutter Documentation - Testing](#)
- [Flutter Documentation - Publishing](#)
- [Mocktail Package](#)